

Verilog Questions

▼ Verilog Interview Questions Set 1

- Write Verilog code to swap contents of two registers with and without a temporary register?
- Elaborate on the file operation support in Verilog.
- Difference between inter statement and intra statement delay?
- What is delta simulation time?
- What is meant by inferring latches and how can you avoid it?
- Which will be updated first - variables or signals ?
- Why is it necessary to list all inputs in the sensitivity list of a combinational circuit ?
- What are the main differences between VHDL and Verilog ?
- Give some examples of commonly used Verilog system tasks and their purposes.
- Write a Verilog code for synchronous and asynchronous reset.

Write Verilog code to swap contents of two registers with and without a temporary register?

Swapping Contents of Two Registers using a Temporary Register:

```
always @(posedge clk) begin
    temp = b;
    b = a;
    a = temp;
end
```

Swapping contents of two registers without a temporary register:

```
always @(posedge clk) begin
    a <= b;
```

```
b <= a;  
end
```

This

is because a non-blocking assignment captures the RHS of all statements in a given delta cycle and assigns them at the end of the cycle. Read more on [Verilog Blocking & Non-Blocking](#) statements.

Elaborate on the file operation support in Verilog.

Verilog

supports file I/O operations - reading from and writing into files.

Verilog file operations work very similar to those of C programming language. Some of the commonly used Verilog system tasks are `$fopen`, `$fscanf`, `$fdisplay`, `$fwrite`, `$fclose`.

Read more on [Verilog File IO Operations](#).

Difference between inter statement and intra statement delay?

Inter

statement delay refers to the delay between two statements. It represents the time difference between the completion of one statement and the start of another statement.

Intra statement delay refers

to the delay within a single statement. It represents the time difference between the start of a statement and the execution of a specific operation within that statement.

Read more on [Verilog Inter and Intra Assignment Delay](#).

What is delta simulation time?

Delta

delay is a special type of delay in Verilog, which is used to model the execution of hardware events that take zero simulation time. It is also known as zero delay.

In Verilog, the delta delay is the smallest delay that can be specified. It represents the smallest unit of

simulation time in Verilog. A delta delay can occur when an event is triggered immediately after the completion of the current statement. In this case, the simulation engine does not advance the simulation time, as there is no actual delay between the two events. Instead, the simulation time stays the same, and the event is executed in a single simulation time step.

Delta delay is used to *model* combinational logic and some types of synchronous logic that do not introduce any delay between input and output. For example, when an input signal changes, a combinational block may immediately process this input and produce an output, with no delay. Read more on [Verilog Scheduling Semantics](#).

What is meant by inferring latches and how can you avoid it?

In Verilog, inferring [latches](#) refers to the unintentional generation of latches when synthesizing an RTL design code. A latch is a sequential logic circuit element that stores a signal value until the next clock cycle. Latches are sometimes undesirable in digital circuit design because they can result in difficult to control circuit behavior, power consumption and unintended glitches.

Inferring latches can occur when a designer does not assign a value to a signal under every possible condition. For example, if a designer creates a combinational logic circuit that assigns a value to a signal only under certain conditions, but does not assign a value to that signal under other conditions, the synthesis tool will infer a latch to store that signal value when it is not being explicitly set by the design. A latch is inferred when the output of combinatorial logic has undefined states, that is it must hold its previous value.

```
// "z" missing in the sensitivity list
always @(x or y)
    out = x & y | z;

// Value of "out" when x=0 is not defined, and hence previous value has
to be held
```

```
always @* begin
  if (x) begin
    out = y & z;
  end
end
```

To

avoid inferring latches, a designer should ensure that every signal in the design has a deterministic value assigned to it under all possible conditions. Using `if-else` constructs or case statements, with a default condition to ensure that every signal is assigned a value, can help to avoid inferring latches. A designer should also avoid inferring latches by writing code that applies a defined value to inputs without the need of latch.

Which will be updated first - variables or signals ?

In

Verilog, signals are updated before variables. Signals are used to represent wires or registers in a design, while variables are used to represent local storage elements in procedural blocks such as `always` blocks or `initial` blocks.

When an event occurs, such as a clock edge, signals are updated first based on the new input values. Then, the updated signal values that are registered and stored internally by the hardware will be directed to variable storage. Therefore, if multiple signals and variables are being updated within the same procedural block, the signals are updated first and then the variables are updated based on the new signal values.

It is important to keep in mind the order of updates of signals and variables while writing the Verilog code. This can help to ensure that the design behaves as expected and can prevent unexpected delays or glitches in the circuit.

Why is it necessary to list all inputs in the sensitivity list of a combinational circuit ?

It

is necessary to list all input signals in the sensitivity list of a combinational circuit in Verilog because the sensitivity list specifies the events that will cause the corresponding always block to execute.

```
always @ (a, b)
begin
    c = a & b;
end
```

This always block is sensitive to the input signals a and b. Whenever either a or b changes, the always block will execute and compute the output c. Else, it may result in a latch as discussed above.

Read more on [Combinational Logic with always](#).

What are the main differences between VHDL and Verilog ?

VHDL

(VHSIC Hardware Description Language) and Verilog are two major hardware description languages used for designing digital circuits. The main differences between VHDL and Verilog are as follows:

- **Syntax:** VHDL uses a verbose syntax that reads like natural language, while Verilog uses a more concise syntax that is similar to C programming language.
- **Data Types:** VHDL has a rich set of data types, including arrays, records, and access types, and provides support for user-defined types. Verilog, on the other hand, has a limited set of data types, including wires and registers.
- **Modeling:** VHDL is more focused on concurrent process modeling, with constructs like processes and components, while Verilog is more focused on gate-level modeling, with constructs like gates and always blocks.
- **Libraries:** VHDL uses a library-based approach to manage components and packages,

while Verilog provides a module-based approach to component instantiation.

- **Testing:** VHDL provides a clear distinction between design and testbench code, with separate files for each, while Verilog allows for testbench code to be intermixed with design code.

Read more on [Verilog syntax](#).

Give some examples of commonly used Verilog system tasks and their purposes.

System

tasks are predefined functions in Verilog that can perform complex operations that would otherwise be difficult to implement in a hardware description language. Some commonly used system tasks include:

\$display, \$monitor, \$strobe	Prints the specified message or value to the console or waveform viewer during simulation.
\$fopen, \$fscanf, \$fclose	File operations to open, scan lines and close files
\$random	Generates a random value within the specified range.
\$stop, \$finish	Simulation termination commands to stop/exit execution
\$readmemb, \$readmemh	Reads memory contents from a file into a Verilog array.

These system tasks can be extremely helpful for debugging, testing, and verification of Verilog programs. Read more on [Verilog Display Tasks](#), [Verilog Math Functions](#) and [Verilog File IO Operations](#).

Write a Verilog code for synchronous and asynchronous reset.

Here are examples of Verilog code for implementing synchronous and asynchronous resets:

Synchronous Reset:

```
module synchronous_reset(  
    input clk,  
    input reset_n,
```

```

//input/output ports
// ...
)

reg [7:0] data;

always @(posedge clk)
begin
    if (!reset_n) //active low reset
        data <= 8'b0;
    else
        data <= //update data based on input ports
end

//output ports
// ...
endmodule

```

Asynchronous Reset:

```

module asynchronous_reset(
    input clk,
    input reset_n,
    //input/output ports
    // ...
)

reg [7:0] data;

always @(posedge clk or negedge reset_n)
begin
    if (!reset_n) //active low reset
        data <= 8'b0;
    else
        data <= //update data based on input ports
end

//output ports

```

```
// ...  
endmodule
```

In

the synchronous reset module, the input reset_n is synchronized with the clock signal using an edge-triggered flip-flop. The reset condition is checked on the rising edge of the clock, and if reset_n is low, the data is reset to 0. Otherwise, data is updated based on input ports.

In

the asynchronous reset module, the input reset_n is checked on both rising and falling edges of the clock, and if reset_n is low, the data is reset to 0 regardless of the current clock state.

▼ Verilog Interview Questions Set 2

- What are HDL simulators ?
- What do you understand by continuous assignment ?
- Can `define be used for text substitution through variable instead of literal substitution ?
- What is the difference between \$setup and \$hold ?
- How can you generate a sine wave using the Verilog coding style ?
- What do you understand by casex and casez statements in Verilog ?
- What is duty cycle ?
- What is verilog \$random ?
- What are different types of delay control ?
- What are parallel threads in Verilog ?

What are HDL simulators ?

HDL (Hardware Description Language) simulators are software tools used in the design and testing of digital hardware. They simulate the behavior of digital circuits written in hardware description languages such as Verilog and VHDL. HDL simulators allow designers to test the functionality, timing, and performance of their designs before

they are implemented in physical hardware. They are essential tools in the design and verification of complex digital systems such as microprocessors, FPGAs, and ASICs. HDL simulators come in different forms, including standalone software tools, integrated development environments (IDEs), and cloud-based platforms.

What do you understand by continuous assignment ?

In

Verilog, a continuous assignment statement allows the designer to assign a value to a signal or a wire continuously as long as the input changes. Unlike procedural assignments, which assign values to signals triggered by an event or a condition, continuous assignments are always active and assign values to signals based on their inputs.

A continuous assignment statement in Verilog is represented by the keyword `assign`

followed by the expression that describes the signal. A continuous assignment is typically used with combinational logic circuits where the output depends solely on the input.

```
// Assigns the logical OR of (a and b) and c to the signal "out"
assign out = (a & b) | c;

// Assigns the logical AND of reset_n and enable_i to "enable"
assign enable = (reset_n & enable_i);
```

Read more on [Verilog assign statement](#).

Can `define be used for text substitution through variable instead of literal substitution ?

Unfortunately, no, the ``define` directive in Verilog does not allow for text substitution through variables.

The ``define`

directive is used in Verilog to define a macro, which is a piece of code that is replaced with a predefined value or string of text during

compilation. The ``define` macro can be used for literal substitutions only, where the pre-defined text is replaced with the actual text value defined.

For example, the following code defines a macro named "DATA_WIDTH" with the value "32":

```
`define DATA_WIDTH 32
```

This macro can be used in the Verilog source code to specify a data width of 32 bits, as shown below:

```
wire [`DATA_WIDTH-1:0] data_bus;
```

During compilation, the ``define` macro is replaced with the pre-defined value "32", resulting in the following code:

```
wire [32-1:0] data_bus;
```

However, the ``define` directive does not allow for variable substitution, and it cannot be used to replace text with variables. Therefore, the pre-defined text value cannot be replaced with variables during compilation.

Also read on [Verilog `ifdef Conditional Compilation](#).

What is the difference between \$setup and \$hold ?

`$setup` and `$hold`

are timing checks used in Verilog to ensure that the input signals are stable at the input of the flip-flop during the setup and hold time windows of the flip-flop.

`$setup` is a timing check used to ensure that the input signal to a flip-flop changes sufficiently ahead of the clock rising edge so that the signal can settle stable by the time the clock edge arrives. `$setup` specifies the minimum time required for the input signal to reach a stable value before the active edge of the clock. Violation of the setup time can cause a race condition, metastability, and unpredictable behavior of the flip-flop.

`$hold` is a timing check used to ensure that the input signal to a flip-flop does not change while the clock is active (high or low) and the input is being sampled by the flip-flop. `$hold` specifies the minimum time that the input signal should be held stable after the active edge of the clock. Violation of the hold time can cause a data loss, timing violations, and unpredictable behavior of the flip-flop.

How can you generate a sine wave using the Verilog coding style ?

Generating

a sine wave using Verilog requires performing some mathematical operations using the Verilog logic. One way to generate a sine wave is by using a lookup table of precomputed sine values and updating the index into the table using a sine frequency and the sampling frequency.

Here is a Verilog code that generates a sine wave with a 16-bit amplitude at 440Hz using a 8-bit lookup table:

```
module sine_wave_generator(  
    input logic clk,  
    input logic reset,  
    output logic signed [15:0] sine  
);  
    // lookup table containing 256 precomputed sine values  
    logic signed [15:0] sin_table [0:255];  
    // initialize the lookup table with sine values  
    initial begin  
        integer i;  
        for (i = 0; i < 256; i++) begin  
            sin_table[i] = $signed(32767 * $sin(2 * $pi * i / 256));  
        end  
    end  
    // sine wave generation  
    logic [15:0] index;  
    logic [7:0] phase_acc;  
    logic [7:0] phase_inc = (440 / 8000) * 256;  
    always @(posedge clk) begin
```

```

    if (reset) begin
        index <= 0;
        phase_acc <= 0;
    end else if (phase_acc >= 255) begin
        phase_acc <= 0;
        index <= index + 1;
    end else begin
        phase_acc <= phase_acc + phase_inc;
    end
    // output the sine value from the lookup table
    sine <= sin_table[index[7:0]];
end
endmodule

```

In

the above code, the sine wave is generated by updating the phase accumulator based on the desired frequency and the system clock frequency. The phase accumulator is used as an index into the lookup table, which contains precomputed sine values scaled to the amplitude of 32767. By outputting the sine value from the lookup table, we get a sine wave output of the desired frequency and amplitude.

What do you understand by casex and casez statements in Verilog ?

The `casez` and `casex` statements are conditional statements in Verilog that compare a case expression against a set of possible matching patterns. Both statements are used to simplify conditional statements by reducing the number of conditional decisions to be made.

The `casex` statement matches the case expression with the patterns that have X in the matching bits. The x bits are automatically treated as don't care bits, and can match either 0 or 1. The remaining bits are treated as exact match bits. The `casex` statement is effective when we have to deal with multi-bit signals that contain unknown or high impedance values where the majority of the bits are either known or known to be zero.

Here's an example of a `casez` statement in Verilog:

```

reg [3:0] my_input;
reg [7:0] my_output;
always @ (my_input) begin
    casez(my_input)
        4'b0000? : my_output = 8'd0;
        4'b0001? : my_output = 8'd1;
        4'b10???0 : my_output = 8'd2;
        default : my_output = 8'hFF;
    endcase
end

```

The `casez` statement above matches the four possible patterns of the input signal `my_input`. The `?` symbol in `my_input` means the value is don't-care, i.e., the input signal can be either 0 or 1 in that position. If the `my_input` signal matches any of the patterns specified, then the corresponding output value is assigned to the `my_output` signal. If `my_input` does not match any of the defined patterns, the default state is executed.

On

the other hand, the case statement uses exact matching logic to compare the case expression to each of the specified patterns. Any bits that are not defined in the specified pattern must match a zero. If the exact matching bits match, the output value assigned to the specified pattern is assigned to the output signal.

Here's an example of a simple case statement in Verilog:

```

reg [3:0] my_input;
reg [7:0] my_output;
always @ (my_input) begin
    case(my_input)
        4'b0000 : my_output = 8'd0;
        4'b0001 : my_output = 8'd1;
        4'b1010 : my_output = 8'd2;
        default: my_output = 8'hFF;
    endcase
end

```

What is duty cycle ?

Duty

cycle is a percentage ratio of the time that a signal is ON compared to the total period of the signal. The duty cycle of a periodic waveform is defined as:

$$\text{Duty cycle} = (\text{time the signal is ON} / \text{total period of the signal}) * 100$$

A signal with 50% duty cycle means that the signal is ON for half of the total period and OFF for half of the total period.

The

concept of duty cycle is used in many electronic devices and circuits, such as pulse width modulation (PWM), signals from oscillators, and digital communication systems. In PWM, the duty cycle of a square wave signal determines the amount of energy that is being delivered to a load, such as a motor or an LED. In digital communication systems, the duty cycle of a signal affects its power consumption and the signal quality.

The duty cycle is an important parameter to consider when designing electronic systems or circuits, as it determines the characteristics and behaviour of the signal. A low duty cycle signal is one that is mostly OFF and has a long period of time between pulses, while a high duty cycle signal is one that is mostly ON and has a shorter period of time between pulses. Understanding the duty cycle of a signal can assist designers to optimize the performance and power consumption of their designs.

What is verilog \$random ?

In Verilog, the `random` system task generates a random value. It is used to simulate unpredictable values in a Verilog testbench. You can use the `$random` task to generate a random value for a signal every time the module is executed, thereby ensuring that the design is tested with different scenarios.

The `$random` system task returns a 32-bit signed integer. The range of values that the `$random` task can generate is dependent on the simulator and the seed value

used. The seed value is an initial value that is used by the random number generator to produce the sequence of random numbers.

Here's an example of how to use the `$random` task in Verilog:

```
module testbench;
  reg [7:0] data;

  initial begin
    $display("Random numbers: ");

    // Generate 32b random numbers ten times and display
    for(int i = 0; i < 10; i++) begin
      $display("%d: %b", i, $random);
    end
  end
endmodule
```

It is worth noting that the `$random` task generates random numbers based on the seed that is set. Hence, to ensure that the simulations are reproducible, it is a common practice to initialize or set/note the seed value for `$random` sequence generator using simulator options.

What are different types of delay control ?

In Verilog, delay control is used to simulate timing delays `within` a module or at the module interface. There are four types of delay control in Verilog:

- The `#delay` delay control is used to add a specified delay to a procedural block. This delay control specifies a delay in timescale units.
- The `@posedge` delay control is used to trigger a procedural block on the positive edge of a clock signal. The procedural block executes after a delay determined by the simulation scheduler.
- The `@negedge` delay control is used to trigger a procedural block on the negative edge of a clock signal.
- The `wait` delay control is used to pause a module's execution for a specified delay value. This delay is specified using timescale units.

Read more on [Verilog Delay Control](#).

In

summary, the different types of delay control in Verilog allow designers to simulate the timing delays that are inherent in digital circuits. By using delay control, Verilog code can more accurately model a digital system's timing characteristics, thereby enabling effective verification of the system's behavior.

What are parallel threads in Verilog ?

In

Verilog, parallel threads refer to code blocks within a module where multiple processes execute concurrently. Each process is a self-contained block of code that runs independently of the other processes. This includes `initial` and `always` blocks.

Parallel threads are also created using the `fork...join` construct in Verilog. The `fork` statement can be used to start multiple parallel threads of execution, while the `join` statement can be used to join the threads back together.

Here's an example code snippet that demonstrates the use of fork and join to create parallel threads:

```
module parallel_threads;
    reg a, b, c, d;

    always @(a or b) begin
        fork
            if (a) begin // Thread #1
                c = 1;
                #10ns;
                c = 0;
            end

            if (b) begin // Thread #2
                d = 1;
                #20ns;
                d = 0;
            end
        join
    end
end
```

```
end  
endmodule
```

In this example, whenever input signals a or b change, a new parallel thread is initiated. Each thread within the `fork...join` block runs independently and concurrently. In this case, the execution of the blocks of code inside the fork statement would overlap with each other, allowing for independent processing of the if statements.

Parallel

threading is particularly useful in testbenches, where multiple processes can be executed concurrently, simulating different parts of a design module.

▼ Verilog Interview Questions Set 3

- What is a defparam used for ?
- What is \$time in Verilog?
- How is rise, fall and turnoff delays represented in Verilog ?
- What is meant by logic synthesis ?
- Give a few examples of compiler directives.
- What are Verilog_parallel case and full case statements ?
- What does a wire refer to?
- What is a reg in Verilog?
- What are blocking and non-blocking statements ?
- Give major differences between a task and a function

What is a defparam used for ?

In Verilog, the `defparam` statement is used to override or set values for module parameters that were declared in the module definition.

When

a module is instantiated, it can have several parameters, such as size or width of a bus, that are declared in the module definition. By default, the parameters are assigned default or predetermined values.

However, sometimes the module might need to be instantiated with different parameter values. In such cases, `defparam` statement can be used to override default parameter values.

Here's an example:

```
module my_module #(parameter WIDTH=8) (  
    input [WIDTH-1:0] data_in,  
    output [WIDTH-1:0] data_out  
);  
// ...  
endmodule
```

In this example, `my_module` has a single parameter, `WIDTH`, that has a default value of 8. When `my_module` is instantiated by default, the `WIDTH` parameter would be set to 8. However, the `defparam` statement could be used to assign a different value to `WIDTH`, as shown:

```
my_module u1 ( .data_in(in_data), .data_out(out_data) );  
  
defparam u1.WIDTH = 16;
```

In this case, a new instance of `my_module`, `u1`, is created with the default `WIDTH` value of 8. However, the `defparam` statement is used to override the default value and set `WIDTH` parameter to 16 for instance `u1`.

Read more on [Verilog Parameters](#).

What is \$time in Verilog?

In Verilog, `$time` is a system task that returns the current simulation time in simulation cycles or time units. It returns a 64-bit unsigned integer value.

The value returned by `$time` is calculated based on the ``timescale` directive in the Verilog code. The ``timescale` directive specifies the simulation time units and time precision used during simulation.

Here's an example of how `$time` can be used in Verilog:

```

module testbench;
    reg clk;

    initial begin
        clk = 1'b0;
        #10;
        while(1) begin
            #5;
            clk = ~clk;
            $display("Sim time is %d", $time);
        end
    end
endmodule

```

In this example, a testbench module with a clock (clk) is defined. The `initial` block sets the clk to 0 and waits for 10 time units. Then, an infinite loop starts toggling the clk signal every 5 time units, and `$display` statement prints the current simulation time (`$time`) to the console.

By

running the simulation, the console output would display the simulation time at each clock edge change. The simulation time increases by 5 time units per iteration. Read more on [Verilog Timescale](#).

How is rise, fall and turnoff delays represented in Verilog ?

In

Verilog, rise, fall, and turnoff delays for digital signals can be represented using different methods, depending on the design specifications and requirements. One common way to represent delays is by using the delay model `#` operator, which specifies a delay in simulation time units. The delay value is specified as a positive integer value preceded by the `#` symbol. For example:

```

wire a, b, out;

nand #5 (out, a, b);

```

In this example, the nand gate has a delay of 5 time units. This means that the output will be generated 5 time units after the input signals change.

Another way to represent delays is by using the `specify` block, which is a construct used to model delay or timing constraints in Verilog. Within a `specify` block, timing paths or delay models can be defined. Here's an example:

```
specify
  specparam delay = 5;
  delay (a, out) = (specify_values ⇒ (delay, 0));
endspecify
```

In this example, a `specify` block is used to define the delay timing path between signals `a` and `out`. The `specparam` statement is used to define the delay parameter with a value of 5. Then, the delay statement is used to specify the timing path between `a` and `out`.

The above code declares that the output (`out`) is delayed by 5 time units from the input signal (`a`).

Alternatively,

the delays can be specified in the gate-level netlist (which can be generated from a higher-level description). The gate-level model can specify delays using the Delay Model Template (DMT).

In summary, rise, fall, and turnoff delays for digital signals in Verilog can be represented using the `#` operator, `specify`

block or Delay Model Template (DMT) in the gate-level. Selecting the appropriate delay representation method depends on the specific requirements of the design being implemented.

What is meant by logic synthesis ?

Logic

synthesis is the process of transforming an electronic circuit design description (specified in a high-level hardware description language like Verilog or VHDL) into a gate-level netlist (specific sequences of AND, OR, NAND, NOR gates, etc.) that can be implemented in hardware.

The synthesis process involves several steps, including technology mapping, optimization, `logic` restructuring, and timing optimization.

During

the technology mapping step, the high-level circuit design is mapped to a set of primitives (like AND, OR, NAND, NOR gates, etc.) provided by the target technology library. The optimization step tries to reduce the complexity of the circuit by simplifying the `logic` using Boolean algebra and other optimization algorithms. The `logic` restructuring stage helps rearrange the circuit layout to further optimize it. Finally, timing optimization ensures that the circuit meets the timing constraints defined in the design.

The end result of the `logic` synthesis process is a gate-level netlist that can be further processed and physically implemented using Electronic Design Automation (EDA) tools, such as place and route tools to optimize the physical layout of the circuit.

Logic synthesis helps in improving the hardware implementation of digital circuits and reduces the design implementation and debugging time. It is a crucial stage in the development process of digital hardware because it forms the foundation for subsequent stages like placement and routing, `static` timing analysis, and formal verification.

Read more on [ASIC Design Flow](#).

Give a few examples of compiler directives.

Compiler

directives are used in computer programming to provide specific instructions to the compiler or preprocessor to process the code in a certain way or to add additional functionality to the code. Here are some examples of compiler directives:

- **``include`**: This directive is used to include another source file in the code. The content of the source file is added to the current file during the pre-processing phase. For instance,

```
`include "interface.sv"
```
- **``define`**: This directive defines a macro, which can be used to represent a value or expression in the code. For instance,

```
`define NUM_MUX_INST 4
```

This will define a macro named NUM_MUX_INST with the value 4.

- **`ifdef / `ifndef:** These directives test whether a certain macro is defined or not. These are useful for creating cross-platform code that works on multiple systems. For instance,

```
`ifdef FEATURE_1
    // some verilog code
`else
    // some other verilog code
`endif
```

Read more on [Verilog `ifdef Conditional Compilation](#).

What are Verilog parallel case and full case statements ?

When

all binary values of the case expression are covered by the case items, the statement is called a full case statement and it avoids inference of latches.

```
// Example of full case with all possible matches
case(abc)
    2'b11 : out3 = 1'b1;
    2'b10 : out2 = 1'b1;
    2'b01 : out1 = 1'b1;
    2'b00 : out0 = 1'b1;
endcase
```

It is a parallel case if the case items are mutually exclusive.

```
// Example of parallel case where items are mutually exclusive
case (abc)
    3'b1?? : out2 = 1'b1;
```

```
3'b01? : out1 = 1'b1;  
3'b001 : out0 = 1'b1;  
endcase
```

What does a wire refer to?

In Verilog, a `wire` is a type of net data type that represents a physical connection between two or more logic gates. It is called a "wire" because it behaves like a real-world electrical wire, allowing signals to flow from one point to another.

Wires are used to connect signals between modules, and can only be driven by a module's output. They are also used internally to connect different logic gates within a module. Wires are meant to represent continuous values, such as analog signals, rather than discrete values like bits.

Read more on [Verilog Data Types](#).

What is a reg in Verilog?

In Verilog, `reg` is a data type used to store and manipulate binary and integer values.

Despite its name, a `reg` does not always represent a physical register or flip-flop. It is a variable that can store a value determined by combinational logic or sequential logic. Its value can be set or reset using an always block or an initial block. Once set, the value can be updated or accessed anytime within the module.

Read more on [Verilog Data Types](#).

What are blocking and non-blocking statements ?

Blocking assignments are represented using the `=` symbol, and they execute sequentially, one after another. The next statement will not start until the current blocking assignment is completed. The value of the right-hand side expression is immediately computed and assigned to the left-hand side variable.

Non-blocking assignments are represented using the `<=` symbol. They appear to execute simultaneously, without waiting for the previous non-blocking assignment to complete. The right-hand side expression is scheduled for execution but the actual assignment to the left-hand side variable is delayed until the end of the time step or clock cycle.

Read more on [Verilog Blocking & Non-Blocking](#).

Give major differences between a task and a function

Tasks and functions are the subroutines used in Verilog. The primary difference is that a `task` can contain statements that consume simulation time like delays, whereas a `function` cannot.

▼ Verilog Interview Questions Set 4

- [What is a sensitivity list ?](#)
- [Explain \\$monitor, \\$display and \\$strobe.](#)
- [What does transport delay mean in Verilog ?](#)
- [What is meant by inertial delay ?](#)
- [What are good practices of writing FSM code ?](#)
- [Explain force and release commands in Verilog.](#)
- [Explain stages in the setup of a regression environment for simulations?](#)
- [What does timescale 1ns/1ps mean?](#)
- [What are some of the features in VHDL?](#)
- [What is PLI ?](#)

What is a sensitivity list ?

In Verilog, a sensitivity list is a list of signals that are used to trigger the execution of a procedural block, such as an `always` block or `initial` block. The sensitivity list determines which signal changes will cause the procedural block to execute.

When

a signal in the sensitivity list changes, the procedural block is

executed. If no signal in the sensitivity list changes, the procedural block is not executed.

The sensitivity list is declared using the `@` symbol followed by a list of signal names in brackets "()", separated by commas. The sensitivity list must include all inputs to the procedural block which are used to operate on, so that any change in these signals will trigger its executable code.

For example, consider the following always block:

```
always @(posedge clock or posedge reset)
begin
    if (reset)
        register <= 0;
    else
        register <= data;
end
```

In the above example, the sensitivity list is "(posedge clock or posedge reset)", which means that the always block will be executed whenever there is a positive edge on the clock signal or a positive edge on the reset signal.

Explain \$monitor, \$display and \$strobe.

In Verilog, \$monitor, \$display, and \$strobe are built-in system tasks used for printing information about the simulation. Here's an explanation of each one:

- **\$monitor** :

\$monitor is a system task used to display variable values every time there's a change in the value of the specified variables. This task takes the form:

```
$monitor(format, list_of_variables);
```

where format specifies the format in which the variables are to be displayed, and list_of_variables

includes the variables whose values should be displayed. When any of the variables in the list_of_variables changes value, the formatted values of all variables listed will be displayed.

For example,

```
integer count = 0;  
$monitor("Count = %d", count);
```

The above code will print the value of count every time it changes.

- `$display` :

`$display` is a system task used to display a message along with the values of specified variables. It takes the form:

```
$display(format, list_of_variables);
```

The format specifies the message to be displayed with optional formatting options, and list_of_variables includes the variables whose values should be displayed in the message.

For example,

```
integer count = 0;  
$display("The value of %d is %d", count, count);
```

The above code will display the message "The value of count is 0".

- `$strobe` :

`$strobe` is a system task used to display a message only once at the end of the timestep. It takes the form:

```
$strobe(message);
```

The message specifies the message to be displayed at the end of the current timestep.

For example,

```
$strobe("Current timestep finished.");
```

The above code will display the message "Current timestep finished." at the end of the timestep.

Read more on [Verilog Display Tasks](#).

What does transport delay mean in Verilog ?

In

Verilog, transport delay is a type of delay that represents the time that it takes for a signal to propagate through a real circuit. It is defined using the `#` symbol, followed by a numerical delay value, in units of time, and then the signal that the delay is applied to. For example:

```
a = #5 b;
```

In this example, the signal a will be assigned the value of signal b after a delay of 5 time units.

Transport

delay models propagate all signals to an output after any input signals change. It is commonly used to model circuit behavior, particularly at the level of gate or module instantiation.

What is meant by inertial delay ?

Inertial

delay is a type of delay modeling in Verilog that is used to simulate signals with a certain level of noise or fluctuations. It is a type of delay with some built-in filtering to account for the fact that not all changes in a signal are necessarily significant or indicative of a real state change.

During the simulation, if an input pulse is shorter than the module delay, it is ignored and does not propagate through the circuit. This filtering helps to simulate the lack of response from the circuit to small fluctuations or noise in the input signal that don't indicate a true state change.

What are good practices of writing FSM code ?

Here are some good practices to follow when writing FSM (Finite State Machine) code in Verilog:

- Use a clear and consistent naming convention for state and output signals to make the code more readable and easier to follow.
- Use enumerations to define the set of states and other parameters associated with the FSM. This approach can help to reduce the risk of errors and increase the readability of the code.
- Avoid using complex expressions or nesting of conditional statements in the logic of the FSM. This can make the code difficult to read, debug, and maintain.
- Write separate blocks of code for the next state logic and output generation. This helps to separate the two different concerns and makes the code easier to read.
- Use one-hot encodings for representing state variables, which can reduce the amount of logic required to implement state transitions and also simplifies debugging, verification, and testing of the FSM.
- Use assertions or simulation-based testing to verify the correctness of the FSM implementation. This helps to ensure that the implementation doesn't have logical errors, and the FSM behaves as expected.

Explain force and release commands in Verilog.

- `force` command is used to force a specific value onto a signal in a simulation until *released*, which overrides any other value that may be set for that variable or signal in the simulation. It does not change the value of the actual signal or variable represented in the hardware, but only changes the value in the simulation.

The syntax of the deposit command is as follows:

```
// "signal" is the signal to set to "value"  
force [signal] = [value];
```

- `release` command is used to allow the signal to resume any other value that may be set for that variable or signal in the simulation.

```
// "signal" is the signal to be released from a prior force  
release [signal];
```

Explain stages in the setup of a regression environment for simulations?

Designing

a regression environment for simulations in Verilog involves several stages, and different coding constructs can be utilized at various stages. The following are some coding constructs of Verilog that can be used during the different stages of designing a regression environment for simulations:

1. **Module Definition:** At the beginning of the design cycle, the first step is to define the various modules that are part of the circuit design. In Verilog, module definitions can be used to build modules for different parts of the circuit design, which can then be used to construct the testbenches.
2. **Testbench Construction:** After module definitions have been created, testbench construction can begin. Testbenches are written in Verilog to simulate different scenarios that the circuit may encounter. Verilog constructs such as "initial" and "always" blocks can be used to define the behavior of the testbench.
3. **Test Vector Generation:** Test vectors are input signals that are designed to stimulate the circuit and detect any malfunctions. Verilog constructs such as "generate" and "for loops" can be used to generate multiple input signal patterns or combinations automatically.

4. **Coverage Collection:** Once the testbench has been constructed and test vectors generated, the next stage is to analyze the coverage of the testbench. Coverage collection is important to ensure that all the possible scenarios are simulated. Verilog constructs such as "coverpoint" and "cross" can be used to define the coverage goals with the scope of the different scenarios.
5. **Result Reporting:** After the simulation regression is completed, the results are analyzed to determine if the circuit design meets the required specifications. Information related to the simulation result is displayed through reporting features, such as Verilog's "assertions," "finish" statement, and "display" statement.

What does timescale 1ns/1ps mean?

In SystemVerilog, the ``timescale` keyword is used to set the units of time for a simulation. The timescale specifies the ratio of simulation time units to real time units.

```
`timescale time_unit/precision_unit
```

Here, time_unit is the base unit of time used for simulation and precision_unit is a unit of measurement for simulation resolution.

For example, if the timescale is set to ``1ns/1ps``, this means that one simulated second is equivalent to one real-time nanosecond (time_unit) and that the simulation resolution is in picoseconds (precision_unit).

This means that within the simulation, any delays or timing constraints specified in the code are treated as being relative to the timescale, i.e., any delay specified as "1" within the code will be interpreted as "1ns/1ps" in real-time units.

The choice of timescale depends on the design and the requirements of the simulation. In general, it's a good practice to use the smallest possible timescale that still satisfies the design requirements, since smaller timescales can improve the accuracy of the simulation.

Read more on [Verilog Timescale](#).

What are some of the features in VHDL?

VHDL is a hardware description language used to model digital circuits and systems. Some of the features of VHDL include:

- **Strong typing:** VHDL is a strongly typed language, which means that every object must be declared with a specific data type before it can be used in the design.
- **Concurrency:** VHDL supports the design of concurrent systems, where multiple processes or threads execute simultaneously within a design. This allows designers to model complex systems with many interacting components.
- **Modularity:** VHDL supports the concept of modular design, which allows designers to create reusable components that can be assembled together to build larger systems. Modules can be instantiated multiple times within a design, making it easier to create more complex circuits.
- **Parameterization:** VHDL allows the creation of modules with parameters that can be set at instantiation time. This helps to create more flexible and reusable designs.
- **Process-based modeling:** VHDL uses processes to describe the behavior of a circuit or system. Processes can be created to model combinational or sequential logic, and can be used to specify complex behaviors within a design.
- **Hierarchical design:** VHDL allows the creation of hierarchical designs, where modules can be instantiated within other modules. This makes it easier to create and manage complex designs by breaking them down into smaller, more manageable parts.
- **Simulation and synthesis:** VHDL supports both simulation and synthesis, which means that a design can be tested and debugged using simulation tools, and then synthesized to create a

physical implementation of the design. This allows designers to develop and optimize a design using simulation tools before committing to a physical implementation.

What is PLI ?

PLI

stands for Programming Language Interface, which is an interface that allows software developers to access and control simulation data within a Verilog or VHDL simulation environment.

The PLI is a set of functions and routines that enable developers to extend the capabilities of the simulation environment by creating new data types, customizing the behavior of primitives, and adding data analysis routines. Essentially, the PLI provides a way for developers to interact with and manipulate the simulation data and results.

The PLI is commonly used in hardware verification and validation, where simulation is used to test the functional behavior and performance of digital circuits and systems. Developers can create custom data analysis routines or data export functions using the PLI to analyze simulation results and generate relevant reports or data exports.

There are several types of PLI interfaces, including the VPI (Verilog Programming Interface), VHPI (VHDL Programming Interface), and DPI (Direct Programming Interface). These interfaces provide different levels of control and flexibility, and can be used for specific applications depending on the requirements of the simulation and the developer's needs.

▼ Verilog Interview Questions Set 5

- Illustrate a few important considerations in Verilog simulation regressions.
- Illustrate the side effect of specifying delays in assign statements.
- Illustrate the side effects of multiple processes writing to the same variable.

- Same variable used in two loops running simultaneously
- Illustrate the side effect of an implicit 1 bit wire declaration of a multi-bit port during instantiation
- Explain what happens when width of state registers is not increased as more states gets added in a state machine.
- What is the purpose of DPI ? Give an example.
- What is a parameter in Verilog?
- Illustrate the side effect of not connecting all the ports during instantiation
- Illustrate the side effect of leaving an input port unconnected that influences a logic to an output port.

Illustrate a few important considerations in Verilog simulation regressions.

Simulation

regressions are a vital part of the design cycle for digital circuits. Simulation regressions involve running a wide range of tests on a circuit design to determine how it behaves under different conditions.

There are several important considerations to keep in mind when executing simulation regressions, including:

1. **Test Coverage:** The goal of a simulation regression is to test the circuit design thoroughly to ensure that it meets the required specifications. Therefore, test coverage is crucial to ensure that all the possible scenarios are simulated.
2. **Scalability:** As the design of a digital circuit becomes more complex, the number of tests required to verify its functionality increases. Therefore, it is essential to ensure that simulation regressions are scalable, and the testbench can be easily modified as per the design.
3. **Debugging Capabilities:** It is essential to have a comprehensive debugging capability to identify the faults encountered during simulation regression.

4. **Simulation Accuracy:** The accuracy of the simulation directly impacts the breadth and depth of test coverage.

Illustrate the side effect of specifying delays in assign statements.

Delays

are not synthesizable and synthesis tools ignore any kind of delays specified in assignment, blocking or non-blocking procedural statements.

If the functionality depends upon the presence of the delay, then a mismatch in functional simulation will be seen between the model and the synthesized netlist.

```
z <= #5 x;    // #5 will be ignored
#10 z <= x;   // #10 will be ignored
```

Illustrate the side effects of multiple processes writing to the same variable.

Some potential side effects of multiple processes writing to the same variable include:

- **Data Races:** Concurrent access to the same variable without proper synchronization can lead to data races. A data race occurs when two or more processes access the same shared variable and at least one of the processes modifies the variable. This can result in unpredictable output or program crashes.
- **Inconsistent Values:** Multiple updates to the same variable by different processes can result in inconsistent data values. For example, one process might read the variable before another process has finished modifying it, resulting in the use of an outdated value.
- **Non-Atomic Updates:** Updating a shared variable is not necessarily an atomic operation, meaning that the update can require several steps to complete. If two or more processes try to update the same variable simultaneously, this can result in partial updates, corrupt data, or race conditions in simulations.

- **Deadlocks:** When multiple processes try to update the same variable in a circular manner, it can result in a deadlock. Deadlock is a situation where two or more processes are waiting for each other to release a resource, but neither process can make any progress.

Most of the linting and synthesis tools can detect this and throw an error.

Same variable used in two loops running simultaneously

The following code will have functional problems as the same variable is used and updated by two concurrent blocks, although it is syntactically correct.

```
module tb;
  integer i; // Same variable updated by different initial blocks

  initial begin
    for (i = 0; i < 5; i = i+1) begin
      #5 $display("Loop#1 : i=%0d", i);
    end
  end

  initial begin
    for (i = 0; i < 10; i = i+1) begin
      #10 $display("Loop#2 : i=%0d", i);
    end
  end
endmodule
```

SystemVerilog allows you to declare the variable within the `for` loop thereby creating two different variables with "local" scope.

```
module tb;
  initial begin
    for (int i = 0; i < 5; i = i++) begin
      #5 $display("Loop#1 : i=%0d", i);
    end
  end
endmodule
```

```

        end
    end

    initial begin
        for (int i = 0; i < 10; i = i++) begin
            #10 $display("Loop#2 : i=%0d", i);
        end
    end
endmodule

```

Illustrate the side effect of an implicit 1 bit wire declaration of a multi-bit port during instantiation

In

hardware description languages, it is common to connect multi-bit ports between modules through wires. However, if the connection is made without explicitly declaring a wire with the correct number of bits, the language may interpret the connection as an implicit 1-bit wire. This can have unintended consequences.

For example, consider a module with a 4-bit output port that is connected to a 4-bit input port of another module. If the connection is declared as follows:

```

module ModuleA(
    output [3:0] out
);

    ModuleB B(.in(out[3:0]));

endmodule

module ModuleB(
    input [3:0] in
);

    // ...

```

```
endmodule
```

Then

the connection between the two modules is made with an implicit 1-bit wire, which means only the least significant bit will be used in the transfer. This can cause unexpected behavior because only one bit of the 4-bit output from ModuleA will be used in ModuleB, which can lead to incorrect results.

To

avoid this issue, it is important to explicitly declare the number of bits in the wire that connects the two modules, as shown below:

```
module ModuleA(  
    output [3:0] out  
);  
  
    wire [3:0] w;  
  
    assign w = out;  
  
    ModuleB B(.in(w));  
  
endmodule  
  
module ModuleB(  
    input [3:0] in  
);  
  
    // ...  
  
endmodule
```

Explain what happens when width of state registers is not increased as more states gets added in a state machine.

In

a state machine, the state register is used to store the current state of the machine. The number of state bits required in the register depends on the number of states in the machine. Forgetting to increase the width of state registers as more states get added in a state machine can lead to unintended consequences.

For example, if the number

of state bits in the register is less than the number of states in the machine, some state bits will be lost. This truncation will cause some states to have the same encoding, resulting in incorrect behavior. This phenomenon is called *state overlap*. State overlap may go unnoticed during simulation or synthesis, but it can cause significant problems in hardware. It can result in incorrect operation, and in worst cases, it may cause the system to fail completely.

```
// BUG! If cur_state is not updated to a 4-bit variable
// there will be functional failure

logic [2:0] cur_state;

...
case(cur_state)
  4'b0100 : // Move into Write phase
  4'b1100 : // Move into Read phase
  ...
```

To

avoid state overlap, the width of the state register should be increased as more states get added in the state machine. The number of bits required for the state register can be calculated by using the formula $2^n \geq \text{number of states}$, where n is the number of bits in the state register.

What is the purpose of DPI ? Give an example.

DPI

stands for Direct Programming Interface, and it is a feature of SystemVerilog that allows communication between the hardware design

being simulated and external software applications. The purpose of DPI is to enable the hardware design being simulated to interact with software modules.

One example of the use of DPI is in co-simulation, where a hardware design is simulated alongside a software application that interacts with it. The software application could be running on a different platform or operating system, and it communicates with the hardware through the DPI interface. This allows software engineers to test their application with the hardware design in a controlled environment.

Another example of the use of DPI is for modeling user-defined protocols or interfaces. A SystemVerilog design can have modules that interact with other modules using DPI, allowing users to implement their own communication protocols or interfaces.

For

instance, let's assume a SystemVerilog design has a module that performs cryptographic operations. The design can have a DPI function that interacts with software modules written in higher-level languages like Python or C, which perform data input/output (I/O) for this design.

In this case, the software modules perform I/O operations such as reading inputs like cryptography keys or plaintext data from the user at runtime, which are fed to the module using the DPI interface. Once the cryptographic module completes its operation, it can return the output to the software through the same interface, allowing for efficient and flexible communication.

What is a parameter in Verilog?

In

Verilog, a parameter is a named constant that is used to simplify the design by allowing commonly used values to be defined once and then used throughout the code. Parameters are defined using the `parameter` keyword and are typically declared at the beginning of the Verilog module.

For example, consider the following Verilog code that defines a simple NAND gate:

```

module nand_gate(output reg y, input a, input b);
    parameter delay = 1; // Delay value for the gate
    always @(a, b) begin
        y = ~(a & b); // NAND gate implementation
        #delay y = y; // Delay the output by the specified time
    end

endmodule

```

In this code, the delay parameter is defined with a value of 1, which is then used to delay the output by one time unit in the `always` block.

Read more on [Verilog Parameters](#).

Illustrate the side effect of not connecting all the ports during instantiation

An unconnected port in a module is called a *dangling port* or *floating port*, and unconnected input ports have high impedance and are evaluated to Z. If the input port is used in `if else` conditions with `==` operator, then it will evaluate to a logical false.

Leaving

a port unconnected can lead to excessive power consumption and unwanted

coupling between nearby signals. Furthermore, it may lead to timing issues, such as setup and hold time violations, which can cause incorrect data transfer or latching.

Read more on [Verilog Module Instantiations](#).

Illustrate the side effect of leaving an input port unconnected that influences a logic to an output port.

An input port that is unconnected will have high impedance or the logic level Z in functional simulation, and synthesis tools will optimize away the logic that propagates beyond a floating input.

```

module X (input a, b, output c, d);
    assign c = a & b;
    assign d = a | b;
endmodule

module Y (input m, n, output o, p);

    X u_x ( .a (),    // input is left floating
           .b (n),
           .c (o),
           .d (p));

endmodule

```

The above example will get synthesized such that m is not connected to any logic within the module Y and b is directly connected to d in X.

▼ Verilog Interview Questions Set 6

- What are the main data types in Verilog ?
- What is Verilog used for?
- What software is used to simulate Verilog code?
- What does RTL mean ?
- Give some guidelines on how combinational and sequential logic should be coded in Verilog
- Give examples of Verilog code that synthesizes into a latch and flipflop.
- What are some ways a race condition can get created, and how can these race conditions be avoided?
- Difference between Mealy and Moore FSM.
- Implement XOR gate using 2:1 MUX
- What are the different looping constructs in Verilog ?

What are the main data types in Verilog ?

In

Verilog, there are several data types that can be used to represent different types of data. The main data types in Verilog include:

- **Wire:** A wire is used for simple connectivity between Verilog modules. It represents a net that can only have one driver and will be used as an output from one module and input to another.
- **Reg:** A reg is used to represent registers or memory elements in a Verilog design. It is used to store and manipulate data within a Verilog module.
- **Integer:** An integer is a data type used to represent signed integers in Verilog. It has a range of -2147483648 to 2147483647.

Read more on [Verilog Data Types](#).

What is Verilog used for?

Verilog

is a hardware description language (HDL) used to design, model, and simulate digital circuits and systems. It is commonly used in the design and verification of integrated circuits (ICs) and field programmable gate arrays (FPGAs) for various applications in communications, consumer electronics, automotive and industrial automation.

Check out [Verilog Tutorial](#).

What software is used to simulate Verilog code?

There

are several software tools that can be used to simulate Verilog code.

Some of the most popular Verilog simulation software tools include:

- **ModelSim:** ModelSim is a popular Verilog simulation and debugging tool developed by Mentor Graphics. It offers a comprehensive solution for designing and verifying digital designs and offers both GUI-based and command-line interfaces.
- **Xcelium:** Xcelium is yet another popular Verilog simulator from Cadence which has a suite of other debugging tools as well.

- **VCS:** VCS is another popular Verilog simulator developed by Synopsys. It offers high-performance simulation and is widely used for complex designs and verification of ICs and FPGAs.
- **Icarus Verilog:** Icarus Verilog is a free and open-source simulator for Verilog designs. It offers fast and efficient simulations and supports both Verilog and SystemVerilog languages.
- **Xilinx Vivado:** Vivado is a popular Verilog simulator and synthesis tool developed by Xilinx. It offers a comprehensive solution for designing and verifying complex digital circuits and systems.
- **Quartus II:** Quartus II is a popular Verilog simulator and synthesis tool developed by Intel (formerly Altera). It offers a comprehensive solution for designing, simulating, and implementing digital circuits and systems using Verilog and VHDL languages.

Overall, the choice of Verilog simulator depends on the specific requirements of the design, such as simulation speed, complexity, and tool compatibility.

What does RTL mean ?

RTL

stands for Register Transfer Level. It is a design abstraction level that represents digital circuits or systems based on the transfer of data between registers. At the RTL level, a digital design is described in terms of registers, combinational logic, and how data is transferred between registers.

RTL is a popular design abstraction level used in digital circuit and system design, commonly considered the "middle" abstraction level that comes after high-level behavioral descriptions, but before gate-level descriptions. Register Transfer Level design is often represented in a Hardware Description Language (HDL), such as Verilog or VHDL.

RTL is used to verify the functionality of the design before it is synthesized into an actual hardware implementation, including full software simulation, hardware emulation, or FPGA prototyping. RTL design is also used in system-level verification and validation, including functional test benching, power analysis and optimizations, and timing analysis. Overall, RTL design is a crucial part of the development of digital systems and plays a crucial role in the design cycle from the abstraction of a design idea all the way to its physical realization.

Give some guidelines on how combinational and sequential logic should be coded in Verilog

- Combinational logic should be written using the "always_comb" keyword. This ensures that the logic is evaluated whenever the inputs change, resulting in more efficient synthesis.
- Sequential logic should be written using the "always_ff" keyword. This ensures that the logic is evaluated only on clock edges, resulting in better timing performance.
- Use descriptive names for signal and variable names to improve code readability and maintainability.
- Use blocking assignments (=) for combinational logic and non-blocking assignments (<=) for sequential logic and to ensure that block dependencies are handled correctly.
- Avoid using latches when coding sequential logic. The use of latches can cause timing issues and make the design difficult to debug.
- Use proper indentation to make the code easier to read and understand.
- Use comments to explain the purpose and functionality of each piece of code.
- Use parameterized modules to create reusable designs that can be easily modified and adapted for different applications.

Give examples of Verilog code that synthesizes into a latch and flipflop.

Latch:

```
module latch(input enable, input data, output reg q);
  always @ (enable, data)
  begin
    if(enable)
      q <= data;
  end
endmodule
```

Flip-flop:

```
module flipflop(input clk, input data, output reg q);
  always @(posedge clk) begin
    q <= data;
  end
endmodule
```

Note:

In a real design, a flip-flop would typically have both rising and falling edge sensitivity and may include additional logic for reset.

What are some ways a race condition can get created, and how can these race conditions be avoided?

A

race condition can be created in a program when multiple processes or threads access shared resources simultaneously, and the relative timing of these accesses is unpredictable. For example, if two threads attempt to modify the same variable at the same time, the result may be unpredictable and depend on the order in which the threads execute.

Some common ways a race condition can occur include:

1. **Shared resource access:** When multiple threads or processes access shared resources such as

files, databases, or network connections without proper synchronization, a race condition can occur.

2. **Memory access:** When multiple threads or processes access shared memory locations without proper synchronization or locking mechanisms, data inconsistencies and race conditions can occur.
3. **Timing issues:** When multiple threads or processes are dependent on the timings of each other or the system, race conditions can occur if the relative timing is unpredictable.

Difference between Mealy and Moore FSM.

Mealy

and Moore State Machines are two types of Finite State Machines (FSM) that have different output behaviors based on the current state and input condition:

Moore: In a Moore state machine, the outputs depend only on the current state of the machine, and not on the input condition. The output of a Moore machine is synchronized with the state transitions and generally lags behind the input. The output is generated at the end of each state.

Example: Consider an elevator in a shopping mall that moves automatically between two floors: Ground floor and the First floor. This output of this FSM does not depend on any input.

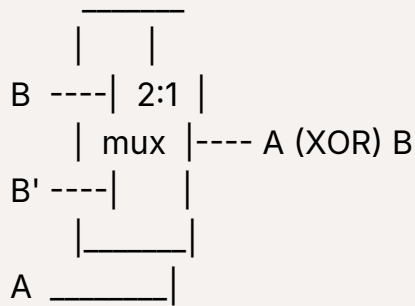
Mealy: In a Mealy state machine, the outputs depend on both the current state and the inputs. The output is generated when the input changes the state. The output changes with every input signal.

Example: Consider a traffic light signal that controls traffic in the north and south direction. The states of the traffic signal are 'red', 'green', and 'yellow'. The traffic signal changes state based on the inputs from the sensors.

Implement XOR gate using 2:1 MUX

A

2:1 Multiplexer (MUX) uses two input signals, one select signal, and one output signal. The output of a 2:1 MUX depends on the value of the select signal. If the select signal is 0, the output will be equal to the first input, and if the select signal is 1, the output will be equal to the second input.



A	B	B'	F
0	0	1	0
0	1	0	1
1	0	1	1
1	1	0	0

▼ Verilog Interview Questions Set 7

- Difference between \$stop and \$finish.
- Design frequency/2 circuit using D flip flop
- What is \$random in Verilog ?
- What is the default value of wire and reg?
- Difference between inter and intra assignment delay.
- Illustrate how the infinite loops get created in the looping constructs like while and for.
- Illustrate the side-effects of a function return type without a range.

- What is #0 in Verilog and its usage?
- How to generate two different clocks in testbench ?
- Implement flip-flop using 2:1 MUX

Difference between \$stop and \$finish.

`$stop`

is used to suspend the simulation at the point where it is called, simulator license is not released and still runs as a process in host operating system. User has to restart it, typically by manually resuming the simulation from the paused point.

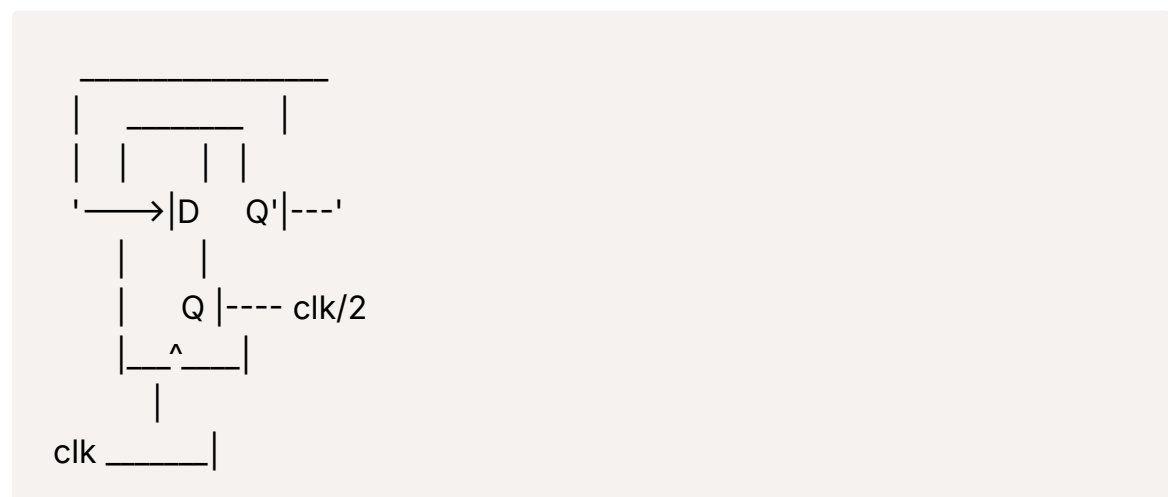
`$finish`

immediately terminates the simulation process and passes control back to the operating system, and license is released because simulation has exited.

`$stop` is useful for debugging and inspection of intermediate results in the design, allowing designers to examine signals, waveforms, or variables at a specific point in execution, while `$finish` is used at the end of the simulation, indicating that the design has completed its operation.

Design frequency/2 circuit using D flip flop

A frequency divider by 2 can be designed using a D flip-flop as follows:



The input clock signal Clk is connected to the D input of the D flip-flop. The output of the flip-flop, Q, is connected back to the D input through an inverter to create a divide-by-2 circuit.

1. When the clock rises from low to high, the D flip-flop captures the value of the D input and outputs it on Q.
2. At the same time, the inverted output Q' feeds it back to the D input.
3. Thus, the output Q changes state on every positive edge of the clock, resulting in a frequency that is half of the input frequency.

Note

that the input clock signal should have a duty cycle close to 50% to ensure reliable operation of the flip-flop. It is also important to ensure that the setup and hold times of the flip-flop are met to avoid timing errors.

What is \$random in Verilog ?

`$random`

is a system task that generates a new 32-bit random integer on every call with a seed value of 0 by default. The optional seed value is used to specify the starting point for the random number generator, and if specified, `$random` generates the same sequence of random numbers every time it is called with the same seed value. It has the following syntax:

```
$random(seed);
```

What is the default value of wire and reg?

In

Verilog, the default value of an uninitialized wire is 'z' or high-impedance and the default value of an uninitialized reg is 'x' or unknown. Read more on [Verilog Data Types](#).

Difference between inter and intra assignment delay.

An intra-assignment delay specifies a delay that occurs within an assignment statement. It is specified using the `#` delay operator immediately following the driver in the assignment statement.

```
assign #5 a = b;
```

An inter-assignment delay specifies the delay between assignments to a signal. It is specified using the `@(delay)` or `wait (delay)` constructs.

```
initial
begin
    @(posedge clk) // This waits for a positive clock edge with a delay of
1 unit
    a = b;
    #2           // This introduces an inter-assignment delay of 2 time units
    a = c;
end
```

Read more on [Verilog Inter and Intra Assignment Delay](#).

Illustrate how the infinite loops get created in the looping constructs like while and for.

Infinite

loops occur in programming when a loop does not have a condition to stop the iteration. Let's take a look at how infinite loops can get created in various looping constructs.

```
reg over;

initial begin
    over = 0;

    while (!over) begin
        #10 $display("[%0t] In Loop", $time);
    end
end
```

The while loop will only exit if some other process changes over to 1, and it will run forever if this does not happen.

Here is another example with `for` loop which will run forever because of a width mismatch in the looping variable. The looping variable i goes from 0 to 7, wraps back to 0, counts up again and never reach the exit loop condition.

```

reg [2:0] i;

initial begin

    for (i = 0; i < 16; i = i+1) begin
        $display("[%0t] i = %0d", i);
    end
end

```

Illustrate the side-effects of a function return type without a range.

If

the range of the function return type is not specified, Verilog assumes it to be a 1 bit scalar value. There would not be any compilation errors but it will result in a functional error.

```

module tb;

    // Should have been function [3:0] add(input [31:0] a, b);
    // Returns 1 bit value by default
    function add(input [3:0] a, b);
        return a + b;
    endfunction

    initial begin
        $display("Sum = %0d", add(4, 5)); // Returns 1
    end
endmodule

```

Read more on [Verilog Functions](#).

What is #0 in Verilog and its usage?

In Verilog, #0

is a delay specifier that represents zero time delay. It is processed after all active events at the current simulation time have been

processed, and its usage is generally not recommended. Like all delays, it is not synthesizable either in designs.

```
module tb;
    reg [3:0] x;

    initial begin
        $monitor("[%0t] x = %0d", $time, x);

        x = 0;
        #10 x = 10;
    end

    initial begin
        #10;
        #0 x = 15;
    end
endmodule
```

In the above code, the #0 delay will make the variable x get value 15 at the end of 10 time units.

How to generate two different clocks in testbench ?

To generate two different clocks in a testbench, you can use two different initial or always

blocks to toggle the two clock signals. Here's an example Verilog code for generating two clock signals with different frequencies using initial blocks:

```
module tb;

    reg clk1;
    reg clk2;

    initial begin
        clk1 = 1'b0;
        forever #10 clk1 = ~clk1; // Generate a clock with 50% duty cycle and
```

```

100ns frequency
end

initial begin
    clk2 = 1'b0;
    forever #5 clk2 = ~clk2; // Generate a clock with 50% duty cycle and
50ns frequency
end

// Rest of the testbench code
endmodule

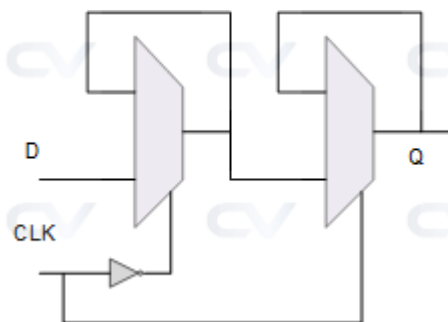
```

In this code, two initial blocks are used to generate two clock signals, clk1 and clk2. The forever loops in each block toggle the clock signal at a specific frequency using the `#` delay operator. You can use these clock signals to drive different components in your design or to test the asynchronous behavior of your design.

Implement flip-flop using 2:1 MUX

A

flip-flop can be implemented using a 2:1 MUX (multiplexer). Here's how you can implement a D flip-flop using a 2:1 MUX in Verilog:



```

module dff_using_mux(input d, clk, output q);
    wire out1;

    assign out1 = ~clk ? d : out1;
    assign q = clk ? q : out1;

endmodule

```

▼ Verilog Interview Questions Set 8

- Give the code for a mod-3 counter
- How can you override the existing parameter value?
- What is Synthesis?
- Write RTL code to generate 60% duty cycle clock.
- Write simple Verilog code to generate 100MHz clock.
- Difference between `define and `include.
- Why always block is not used inside a program block?
- What is a FIFO and write Verilog code for the design?
- What will happen if there is no else part in if-else ?
- Explain overflow and underflow conditions in FIFO.

Give the code for a mod-3 counter

A modulo-3 counter has 3 states (0, 1, 2) and requires 23 number of flip-flops.

```
module cntr_mod3 (input clk, rstn, output reg [1:0] out);
  always @(posedge clk) begin
    if (!rstn)
      out <= 0;
    else
      if (&out)
        out <= 0;
      else
        out <= out + 1;
  end
endmodule
```

How can you override the existing parameter value?

In Verilog, you can override the existing parameter value in two ways:

1. In module instantiation

```
module abc (input ..., output ...);  
    parameter RESET_VAL = 4;  
endmodule  
  
module xyz ();  
    abc u_abc #(.RESET_VAL (10)) ( ... );  
endmodule
```

1. Using defparam

```
module abc (input ..., output ...);  
    parameter RESET_VAL = 4;  
endmodule  
  
module xyz ();  
    abc u_abc ( ... );  
  
    defparam u_abc.RESET_VAL = 10;  
endmodule
```

Read more on [Verilog Parameters](#).

What is Synthesis?

Synthesis

is the process of converting a high-level hardware description language (HDL) code, such as Verilog or VHDL, into a gate-level netlist that can be used for physical implementation of a digital circuit on an integrated circuit (IC) or field-programmable gate array (FPGA).

The

synthesis process involves analyzing the HDL code to determine the intended functionality of the circuit, optimizing the design for the desired performance and resource utilization, and generating a gate-level netlist that describes the circuit in terms of **logic** gates and flip-flops.

The synthesis tool analyzes the HDL code and performs a series of transformations to optimize the design. This can involve simplifying logic expressions, removing redundant logic, optimizing resource usage, and mapping the design to a specific target technology, such as an FPGA or ASIC. The synthesis tool then generates a gate-level netlist that can be processed by other tools to perform place-and-route, to create a physical layout of the circuit, and finally to generate the programming files that can be used to program the target device.

Read more on [ASIC Design Flow](#).

Write RTL code to generate 60% duty cycle clock.

To generate a 60% duty cycle clock using RTL code, we can use a counter to divide the input clock signal and use the output of the counter as the inverted output of a D-type flip-flop.

```
module clock_60_percent (
    input wire clk,
    input wire rst_n,
    output wire clk_60_percent
);

    // Define the maximum count value.
    // For a 60% duty cycle, we want the high time to be 3/5 of the period.
    // If the counter goes from 0 to 4 (5 states), we can have 'high' for 3 states and 'low' for 2 states.
    // So, MAX_COUNT = 4.
    localparam MAX_COUNT = 4;

    reg [2:0] counter; // Needs enough bits to represent MAX_COUNT

    // Duty cycle logic
    assign clk_60_percent = (counter < MAX_COUNT - 1) ? 1'b1 : 1'b0;
    // If counter is 0, 1, 2, 3 (4 states) → HIGH
    // If counter is 4 (1 state) → LOW
    // This gives a 4/5 duty cycle. Let's adjust this.
```

```

// Let's refine this for 60% (3/5).
// We want the output to be HIGH for 3 counts and LOW for 2 counts.
// So, counter values 0, 1, 2 are HIGH. Counter values 3, 4 are LOW.
// This means the counter should go up to 4 (5 states total).
// The condition for HIGH would be counter < 3.

localparam HIGH_THRESHOLD = 3; // Counter values 0, 1, 2 will be HIGH

assign clk_60_percent = (counter < HIGH_THRESHOLD) ? 1'b1 : 1'b0;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        counter <= 3'b0;
    end else begin
        if (counter == MAX_COUNT) begin
            counter <= 3'b0; // Reset counter
        end else begin
            counter <= counter + 1;
        end
    end
end
endmodule

```

Write simple Verilog code to generate 100MHz clock.

To generate 100M clock with 50% duty cycle, we have to flip the clock bit every 5ns.

```

module tb;
    reg clk;

    // 1/100M = 10ns period
    always #5 clk = ~clk;
endmodule

```

Difference between ``define` and ``include`.

- **``define`**

It

is a preprocessor keyword that is used to define a text macro. It can be used to define constants, expressions or strings that can be used later in the code. When the Verilog/SystemVerilog compiler encounters a ``define` directive, it replaces the defined text with the macro value throughout the code before compilation.

```
`define MY_VALUE 4
always @(posedge clk) begin
    if (i == `MY_VALUE) begin
        // some code
    end
end
end
```

Here, MY_VALUE is a macro that has been defined with a value of 4. The macro is being used inside an always block to check if i is equal to 4. When this code is compiled, the preprocessor will expand the MY_VALUE macro into the actual value of 4.

- **``include`**

It

is also a preprocessor directive that is used to include a file in the current code. When the Verilog/SystemVerilog compiler encounters an ``include` statement, it reads the contents of the specified file and inserts them into the current source file at the location of the ``include` statement. The ``include` statement is mainly used to insert common code, libraries or modules into multiple files without copying the same code multiple times.

```
`include "my_file.v"
module my_module (input clk, input rst, output reg out);
    // some code
endmodule
```

Here, the ``include` statement is used to include the contents of the file ``my_file.v` into the current module. The code inside `my_file.v` could contain module definitions, declarations, ``define` statements or other common code that can be reused in multiple modules.

Why always block is not used inside a program block?

A `program`

`block` is used in SystemVerilog to encapsulate design hierarchy and to allow for a more modular design approach. It is a static region of code and cannot contain any behavioral constructs. An `always` block, on the other hand, is used to describe behavioral functionality in a module or an interface.

Since a `program`

`block` cannot contain any behavioral constructs, it cannot contain an `always` block. The only constructs that are allowed inside a `program` block are declarations, instantiations of other modules or interfaces, and control statements such as `if-else`, `case`, `for`, and `while` loops.

What is a FIFO and write Verilog code for the design?

FIFO

(First-In-First-Out) is a type of buffer used in digital circuits to manage data transfer between devices or subsystems. It is a common design element in communication interfaces and memory management units. A

FIFO stores data in a queue-like structure, where the first item that has been inserted into the buffer will be the first one to be removed.

Read more on [Synchronous FIFO](#).

What will happen if there is no else part in if-else ?

If there is no `else` part in an `if-else` statement, then execution will simply move on to the next statement after the `if-else` block, without executing any code when the `if` condition is false.

For example, consider the following Verilog code:

```
always @(posedge clk) begin
    if (data_ready) begin
        data_valid <= 1;
    end
end
end
```

Note that this is a typical error that synthesizes into a latch because it does not specify what has to be done to `data_valid` when `data_ready` is low, and hence it simply holds onto the previous value.

Explain overflow and underflow conditions in FIFO.

In

a FIFO (First-In-First-Out) buffer, overflow and underflow occur when data is being added to or removed from the buffer and there is no space available to add new data (in the case of overflow) or no data available to remove (in the case of underflow).

Overflow occurs when new data is being written into the FIFO while it is full. This means that the buffer has reached its maximum capacity and cannot store any more data. In an overflowing FIFO, new data may overwrite the old data that is already in the FIFO, leading to data loss or corruption. This can cause serious issues in systems where data integrity is critical. To avoid overflow, a flag indicating whether the FIFO is full may be used.

Underflow,

on the other hand, occurs when data is being read from the FIFO while it is empty. This means that there is no data available to remove from the buffer. When underflow occurs, the FIFO may return an invalid value or in some cases, return no value at all. To avoid underflow, a flag indicating whether the FIFO is empty may be used.

To minimize the impact of overflow and underflow, FIFOs are typically designed with a certain amount of headroom or slack, which is the difference between the maximum capacity of the FIFO and the actual data being stored. This allows the FIFO to accept some extra data without overflowing, and also allows for some data to be removed even if the buffer is not completely full. It's important to carefully choose the capacity of the FIFO to

ensure that it can handle the maximum expected data rate in the system, while also providing enough slack to avoid overflow and underflow.

▼ Verilog Interview Questions Set 9

- What are all different applications of FIFO?
- How can you define strength in Verilog
- Design divide-by-5 module.
- Write a Verilog code for 5:1 MUX
- Elaborate on the Verilog event scheduler?
- Difference between dual port ram and FIFO.
- Write Verilog code to store fractional number
- What are synchronous and asynchronous resets ?
- Explain the concept of time borrowing

What are all different applications of FIFO?

FIFOs

(First-In-First-Out) are used in a wide range of applications where data needs to be buffered or stored temporarily. Some of the most common applications of FIFOs include:

- **Memory and data buffering:**

FIFOs are commonly used for buffering data in memory or I/O controllers to ensure a smooth flow of data between different systems or devices.

- **Network routing and switching:**

In networking equipment such as routers and switches, FIFOs are used to store packets of data from different sources and route them to their destination in the correct order.

- **Multimedia applications:**

In multimedia applications such as audio and video processing, FIFOs are used to buffer data streams to ensure smooth playback without any glitches or interruptions.

- **Real-time applications:** In

real-time systems such as industrial automation and control systems, FIFOs are used to synchronize the flow of data and ensure that it is processed in real-time without any delays.

- **Data acquisition:**

FIFOs are used in data acquisition systems to temporarily store data from sensors, ADCs, and other data sources before it is processed or

transmitted.

- **Graphics processing:** In graphics processing units (GPUs), FIFOs are used to temporarily store data such as rendering commands and pixel data before it is processed and displayed on a screen.

Overall, FIFOs have a wide range of applications in different fields where data needs to be stored and processed in a controlled and efficient manner.

How can you define strength in Verilog

In

Verilog, strength is a measure of the signal's electrical characteristics. It indicates how strongly the signal is driven or resisted by the driver. Verilog defines two types of signal strengths, which are:

- **Charge Strength** : This is used only with `triereg` nets and is used to model charge storage which specifies the relative size of the capacitance i.e. small, medium or large.
- **Drive Strength** : This indicates the strength of the logic values on the output terminals of the gate instance, and can be of *strength1* specified by `supply1` , `strong1` , `pull1` and `weak1` or *strength0* specified by `supply0` , `strong0` , `pull0` and `weak0` .

Design divide-by-5 module.

A

divide-by-5 module takes an input clock signal and produces an output that is 1/5th the frequency of the input signal. Design a mod-5 counter, and take the output from the flop that is high for 2 clocks and feed it to a negative edge triggered FF. Then do a logical OR of the delayed version with the original to get the required clock output.

```
module clk_div5 (input clk, rstn, output clk50);
  wire clkA;
  reg clkB;
  reg [2:0] count;

  always @(posedge clk or negedge rstn) begin
    if (!rstn)
      count <= 0;
    else if (count == 4)
      count <= 0;
    else
      count <= count + 1;
  end
endmodule
```

```

end

assign clkA = count[1];

always@(negedge clk)
    clkB <= clkA;

assign clk50 = clkA | clkB;
endmodule

```

Write a Verilog code for 5:1 MUX

A

5:1 MUX selects one of five input signals and forwards it to the output based on the value of two select signals. A possible Verilog implementation for a 5:1 MUX is:

```

module mux_5to1 (input [4:0] data, [2:0] sel, output reg out);

    always @ (data, sel) begin
        case (sel)
            5'h0 : out = data[0];
            5'h1 : out = data[1];
            5'h2 : out = data[2];
            5'h3 : out = data[3];
            default : out = data[4];
        endcase
    end
endmodule

```

The data input is a 5-bit wide vector containing the five input signals to the MUX. The sel input is a 2-bit wide vector containing the two select signals. The out output is the selected input signal.

The always block describes a combinational logic that selects the appropriate input signal based on the value of the sel vector. The case statement selects the input signal corresponding to the value of sel. This implementation assumes that there is a default value for the output signal when sel is not one of the valid selection codes.

Elaborate on the Verilog event scheduler?

The

Verilog event scheduler is an essential part of the Verilog simulator, responsible for scheduling and executing events during the simulation. Events in Verilog can be either timed or procedural.

Timed events

represent changes in the simulation time and are scheduled by the simulator's built-in timing mechanism. The scheduler maintains a list of timed events that are sorted by their scheduled time. When the

simulator advances to a new time, the scheduler looks at the list of timed events and executes any events that are scheduled to occur at the current time or earlier. Timed events can be scheduled using the `#` delays or using statements such as `@` or `wait`.

Procedural events represent changes in the simulation state that are triggered by procedural statements such as `always`, `initial`, and `assign`.

These events are scheduled based on the hierarchy and sensitivity lists of the modules in the design. Whenever a procedural event is triggered, the scheduler adds the corresponding changes to the list of timed events.

Read more on [Verilog Scheduling Semantics](#).

Difference between dual port ram and FIFO.

Dual

Port RAM and FIFO (First-In-First-Out) are both memory elements used in digital designs, but they have some key differences in their architecture, functionality, and use cases.

- **Architecture:**

Dual Port RAM has two separate ports, one for read and one for write access, allowing simultaneous access to different addresses in the memory. FIFO, on the other hand, is implemented as a circular shift register with two pointers (Read and Write pointers) which indicate when the FIFO is full or empty.

- **Functionality:** An address in

Dual Port RAM can be accessed in consecutive cycles whereas a FIFO does not have an address and hence data at the same location can be written or read after a wrap around.

- **Timing control:** Dual Port

RAM allows simultaneous access to different locations, but it requires proper timing control to avoid issues like read-after-write hazards and write-after-write clashes. FIFOs, on the other hand, are optimized for timing control and provide built-in circuitry to detect overflow and underflow conditions.

- **Handshaking:** Dual Port RAM provides control signals like Read/Write enable signals, address signals, and output data signals for handshaking between different modules, whereas FIFOs provide control signals like Read/Write pointer signals and status signals to detect buffer overflow/underflow conditions.

In

summary, Dual Port RAM is a more general-purpose memory element that supports simultaneous access to different memory locations, while FIFO is a specialized memory element that is best suited for buffering and rate conversion between devices with different data rates or clock domains.

Write Verilog code to store fractional number

In

Verilog, fractional decimal numbers are typically stored as fixed-point numbers, where a fixed number of bits are allocated to represent the integer and fractional parts of the number. One common format for fixed-point numbers is $Q_m.n$, where $m+n$ is the total number of bits and n is the number of bits allocated to represent the fractional part.

Here's an example Verilog code to store a Q7.4 fixed-point number:

```
module fixed_point_example();

    reg [6:0] integer_part;    // 7 bits for integer part
    reg [3:0] fractional_part; // 4 bits for fractional part

    reg [10:0] fixed_point_number; // 11 bits for Q7.4 number

    initial begin
        integer_part = 5;
        fractional_part = 8;    // This is 0.5 in Q7.4 format

        fixed_point_number = {integer_part, fractional_part}; // Concatenate integer and fractional parts
        $display("Fixed-point number = %d.%d", fixed_point_number[10:4], fixed_point_number[3:0]); // Display as
        decimal number
    end

endmodule
```

In

this code, the reg declarations define three registers to store the integer part, fractional part, and fixed-point number. The integer_part register has 7 bits to represent the integer part of the number, and the fractional_part register has 4 bits to represent the fractional part of the number in Q7.4 format.

What are synchronous and asynchronous resets ?

Most practical sequential elements require a *reset* signal to have a known initial state on startup.

Asynchronous

reset forces the output low immediately. It requires gating both the data and the feedback to force the reset independent of the clock.

```
// "rstn" added to the sensitivity list so that it evaluates the
// if condition immediately on the negedge of "rstn". At the negedge
// of rstn, its value is 0.
always @ (posedge clk and negedge rstn) begin
    if (rstn) begin
        out <= 0;
    end else begin
        // rest of the logic
    end
end
```

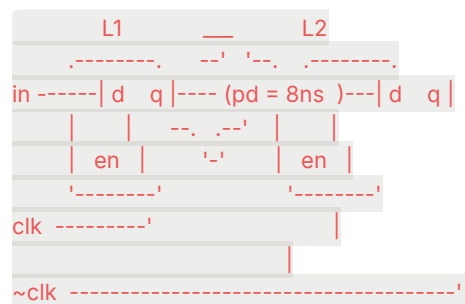
Synchronous reset waits for the clock to force the output low, and it simply needs an AND between the data input and reset signal.

```
// Here, the if condition is evaluated only on the posedge of "clk"
// and the block is reset if "rstn" is found to be 0
always @ (posedge clk) begin
    if (rstn) begin
        out <= 0;
    end else begin
        // rest of the logic
    end
end
```

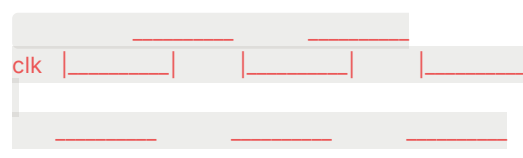
Explain the concept of time borrowing

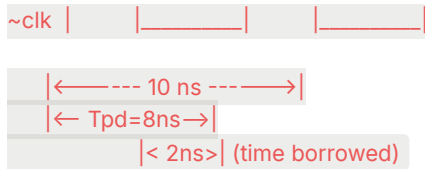
Time

borrowing is a mechanism in which a latch based design effectively uses the transparency between two adjacent latches to meet the propagation delay between them.



Enables for the two latches shown above are opposite in polarity. L1 is enabled during the ON period of clk while L2 is enabled during the OFF period of clk. Assume that the delay for $d \rightarrow q$ and $en \rightarrow q$ is 0ns, the propagation delay for the combinational cloud between the latches is 8ns, and the clock period is 10ns. Time borrowed is 2ns.





- • What are all different applications of FIFO?
- • How can you define strength in Verilog
- • Design divide-by-5 module.
- • Write a Verilog code for 5:1 MUX
- • Elaborate on the Verilog event scheduler?
- • Difference between dual port ram and FIFO.
- • Write Verilog code to store fractional number
- • What are synchronous and asynchronous resets ?
- • Explain the concept of time borrowing
- • **Memory and data buffering:** FIFOs are commonly used for buffering data in memory or I/O controllers to ensure a smooth flow of data between different systems or devices.
- • **Network routing and switching:** In networking equipment such as routers and switches, FIFOs are used to store packets of data from different sources and route them to their destination in the correct order.
- • **Multimedia applications:** In multimedia applications such as audio and video processing, FIFOs are used to buffer data streams to ensure smooth playback without any glitches or interruptions.
- • **Real-time applications:** In real-time systems such as industrial automation and control systems, FIFOs are used to synchronize the flow of data and ensure that it is processed in real-time without any delays.
- • **Data acquisition:** FIFOs are used in data acquisition systems to temporarily store data from sensors, ADCs, and other data sources before it is processed or transmitted.

- • **Graphics processing:** In graphics processing units (GPUs), FIFOs are used to temporarily store data such as rendering commands and pixel data before it is processed and displayed on a screen.
- • **Charge Strength :** This is used only with `triereg` nets and is used to model charge storage which specifies the relative size of the capacitance i.e. small, medium or large.
- • **Drive Strength :** This indicates the strength of the logic values on the output terminals of the gate instance, and can be of *strength1* specified by `supply1` , `strong1` , `pull1` and `weak1` or *strength0* specified by `supply0` , `strong0` , `pull0` and `weak0` .
- • **Architecture:** Dual Port RAM has two separate ports, one for read and one for write access, allowing simultaneous access to different addresses in the memory. FIFO, on the other hand, is implemented as a circular shift register with two pointers (Read and Write pointers) which indicate when the FIFO is full or empty.
- • **Functionality:** An address in Dual Port RAM can be accessed in consecutive cycles whereas a FIFO does not have an address and hence data at the same location can be written or read after a wrap around.
- • **Timing control:** Dual Port RAM allows simultaneous access to different locations, but it requires proper timing control to avoid issues like read-after-write hazards and write-after-write clashes. FIFOs, on the other hand, are optimized for timing control and provide built-in circuitry to detect overflow and underflow conditions.
- • **Handshaking:** Dual Port RAM provides control signals like Read/Write enable signals, address signals, and output data signals for handshaking between different modules, whereas FIFOs provide control signals like Read/Write pointer signals and status signals to detect buffer overflow/underflow conditions.

[Interview Questions](#) [Verilog Interview Set 1](#) [Verilog Interview Set 2](#)
[Verilog Interview Set 3](#) [Verilog Interview Set 4](#) [Verilog Interview Set 5](#)
[Verilog Interview Set 6](#) [Verilog Interview Set 7](#) [Verilog Interview Set 8](#)
[Verilog Interview Set 9](#) [Verilog Interview Set 10](#) [Verilog Interview Set 11](#)
[Verilog Interview Set 12](#) [Verilog Interview Set 13](#) [Verilog Interview Set 14](#)
[Verilog Interview Set 15](#) *Poll How would you rate the overall quality of this website ?* ★ ★★ ★★★★★ ★★★★★★ Related
[Topics](#) [Digital Fundamentals](#) [Verilog Tutorial](#) [Verification](#) [SystemVerilog Tutorial](#) [UVM Tutorial](#) [How can we improve ?](#)

▼ Verilog Interview Questions Set 10

- What logic is inferred when there are multiple assign statements targeting the same wire for synthesis ?
- What do conditional assignments get inferred into?
- What is the logic that gets synthesized when conditional operators in a single continuous assignment are nested?
- What value is inferred when multiple procedural assignments made to the same reg variable in an always block?
- Why should a nonblocking assignment be used for sequential logic, and what would happen if a blocking assignment were used?
- What does the logic in a function get synthesized into? What are the area and timing implications of calling functions in RTL?
- What are a few important considerations while writing a Verilog function?
- What does the logic in a task get synthesized into? Explain with an example.
- What are the differences between using a task, and defining a module for implementing reusable logic?
- Can tasks and functions be declared external to the scope of module-endmodule?

What logic is inferred when there are multiple assign statements targeting the same wire for

synthesis ?

The synthesis tool will give a syntax error for a `wire` that is an output port of a module if it is driven by more than one source.

```
wire out;

assign out = a & b;

// Elsewhere in the code, another assign to
// the same wire will cause multiple driver error
assign out = a | b;
```

However, it is okay to drive a 3-state wire by multiple assign statements.

```
wire out;

// sel1 and sel2 cannot be 1 at the same time
assign out = sel1 ? a & b : 1'bz;
assign out = sel2 ? a | b : 1'bz;
```

What do conditional assignments get inferred into?

Conditional assignments `?:`

in Verilog get inferred into multiplexers during synthesis. A multiplexer selects one of a number of inputs based on the values of the select inputs.

```
// Assign in0 to out if sel = 1 else in1
assign out = sel ? in0 : in1;
```

What is the logic that gets synthesized when conditional operators in a single continuous assignment are nested?

In

Verilog, when conditional operators in a single continuous assignment are nested, the synthesis tool will infer a hierarchy of multiplexers.

```
assign out = sel1 ? (sel2 ? in3 : in4) : (sel3 ? in5 : in6);
```

```
// Which is the same as  
wire net1, net2;
```

```
assign net1 = sel2 ? in3 : in4;  
assign net2 = sel3 ? in5 : in6;  
assign out = sel1 ? net1 : net2;
```

What value is inferred when multiple procedural assignments made to the same reg variable in an always block?

When multiple procedural assignments are made to the same `reg` variable in an always block, the last assignment will be inferred as the final value of the `reg` variable.

```
reg [3:0] data;  
  
always @ (posedge clk) begin  
    data <= 4'hA;  
    data <= 4'h2;  
end
```

During simulation, the data variable will be initialized to an unknown value. When a positive edge of clk occurs, both the assignments will occur in the order they are written. However, since both assignments use non-blocking assignment, the value of data after the clock edge will be the value assigned last, in this case 2.

Why should a nonblocking assignment be used for sequential logic, and what would happen if a blocking assignment were used?

A

nonblocking assignment should be used for sequential logic in Verilog because it models a flip-flop's behavior more accurately. Nonblocking

assignments infer a level-sensitive behavior where the assigned value is used in the next cycle, representing the clock-to-Q delay of the flip-flop.

```
reg a, b, c, out;

always @ (posedge clk) begin
    a <= in1;
    b <= a;
    c <= b;
    out <= c;
end
```

The code above will get synthesized into a single flip-flop with the d input of in1 and q output of c.

This is because the intermediate results were stored in a blocking format, and the final result didn't require waiting for these results to be assigned. Since out is influenced only by the posedge of clock, it turned out to be a FF.

```
reg a, b, c, out;

always @ (in1) begin
    a = in1;
    b = a;
    c = b;
    out = c;
end
```

Assignments

above are made in a combinatorial block using blocking statements and the logic synthesized will be a simple direct connection between in1 and out.

What does the logic in a function get synthesized into? What are the area and timing implications of calling functions in RTL?

The

logic in a function in Verilog typically gets synthesized into combinational logic in the generated hardware since it does not have any constructs that advance time. The function definition essentially defines a block of combinational logic that takes some input values and produces an output value based on those input values.

The area and

timing implications of calling functions depend on the complexity of the function implementation, the number of input and output ports, and the frequency and timing constraints of the system. If the calls to a function are used in different paths, the logic gets replicated, else it may get multiplexed.

```
module des (input [3:0] a, b, opc, output [3:0] out1, out2, out3);

    function [3:0] compute(input [3:0] a, b, opc);
        case (opc)
            2'b00 : alu = a & b;
            2'b01 : alu = a | b;
            2'b10 : alu = a ^ b;
            default : alu = ~(a & b);
        endcase
    endfunction

    assign out1 = compute(a, b, 0);
    assign out2 = compute(a, b, 1);
    assign out3 = compute(a, b, 2);

endmodule
```

In

the example given above, all outputs use has a different use of the function and would get independent logic for each function call. But say if out1 and out2 both required AND gate functionality, then it would use common logic.

What are a few important considerations while writing a Verilog function?

1. **Function interface:** The inputs and outputs of the function should be carefully defined, along with their data types and bit widths. If the width of the return value is not defined, it will end up with a default of 1 bit.
2. **Function scope:** Verilog functions are typically defined within a module, and their scope is limited to that module. As such, it is important to ensure that the function is not trying to access variables or signals outside its scope.
3. **Function complexity:** The complexity of the function implementation should be kept in check, as this can have a significant impact on the area and timing of the generated hardware. Functions should be designed to be as simple and straightforward as possible, avoiding any unnecessary complexity.
4. **Combinational vs Sequential logic:** Verilog functions are intended to be used for combinational logic, however, it is important to ensure that the function does not create sequential logic by introducing latches or flip-flops in the generated hardware.

```
function oh_latch(input in, sel);  
  
    // else part is missing so output will be  
    // latched to "in"  
    if (sel)  
        oh_latch = in;  
endfunction
```

1. **Parameters:** Parameters and integers are local only to that function and cannot be outside its scope.

What does the logic in a task get synthesized into? Explain with an example.

Synthesis tools ignore all timing constructs within a task like `@`.

A task may be used to synthesize basic combinatorial logic, however if

the output of the task is assigned to a storage element, it will synthesize a sequential element.

```
task add_two_inputs(input [7:0] a, input [7:0] b, output [7:0] sum) begin
    sum = a + b;
end

module top;
    reg [7:0] a, b;
    wire [7:0] sum;

    always @(a, b)
        add_two_inputs(a, b, sum);

endmodule
```

What are the differences between using a task, and defining a module for implementing reusable logic?

1. **Hierarchical design:** Modules can be used to implement hierarchical designs by instantiating other modules inside them. Tasks, however, cannot be reused in this way, because they are not self-contained and require a parent module to be called from.
2. **Complexity:** Modules can be used to implement complex designs with significant functionality, and can include sub-modules to make the design more modular and easier to manage. Tasks, on the other hand, are generally used for simpler calculations or tasks that are performed repeatedly throughout the design.
3. **Floorplanning:** Can be placed as a block during floorplanning because it has a hierarchy that is fully defined. But logic inside a task cannot be moved around because it will be just a part in the sea of gates.

4. **Testing:** Modules are often easier to test than tasks because they can be connected to test benches that generate inputs and check outputs. Tasks can be tested, but it requires creating a parent module to call the task and then testing that module.

Can tasks and functions be declared external to the scope of module-endmodule?

Yes, tasks and functions can be declared external to the scope of `module` `endmodule` in Verilog for versions after 2001.

▼ Verilog Interview Questions Set 11

- What are the considerations to be taken choosing between flop-flops vs. latches in a design?
- Which one is better, asynchronous or synchronous reset for the storage elements?
- What logic gets synthesized when integer is used instead of a reg variable as a storage element?
- How do I choose between a case statement and a multi-way if-else statement?
- How do I avoid a priority encoder in an if-else tree?
- What are the differences between if-else and the ?: conditional operator?
- What is the importance of a default clause in a case construct?
- What is the difference between full_case and parallel_case synthesis directive?
- What is the difference in implementation with sequential and combinatorial processes, when the final else clause in a multiway if-else construct is missing?
- What is the difference in using (== or !=) vs. (=== or !==) in decision making of a flow control construct in a synthesizable code?

What are the considerations to be taken choosing between flop-flops vs. latches in a design?

1. Functional requirements:

The first consideration is the functional requirements of the system. Different types of flip-flops and latches have their specific functions and capabilities, and it is necessary to choose the appropriate type based on the system's requirements.

2. Timing requirements:

Flip-flops have a fixed clock edge at which they latch the data, while latches hold the data as long as the enable signal is active. Latches facilitate time borrowing or cycle stealing, and helps increase pipeline depth with lesser area.

3. Power consumption: Flip-flops

tend to consume more power than latches, as they operate continuously with a clock signal. If power consumption is a concern, latches may be preferred.

4. Noise immunity: Flip-flops are more immune to noise than latches since they latch their values at a specific clock edge. If the design has a lot of noise, flip-flops may be the better option.

5. Area and cost: Latches are typically smaller and less expensive than flip-flops. If the design needs to optimize for area or cost, latches may be preferred.

6. Operating frequency:

With time borrowing and cycle stealing, operating frequency is higher than the slowest logic path for latch. But in the case of FF, the slowest path pretty much decides the operating frequency.

Read more on [Sequential Logic](#).

Which one is better, asynchronous or synchronous reset for the storage elements?

- Reset signal is not part of the data path in async-reset whereas reset signal is part of the D input of the FF in sync-reset.
- Effect of reset can happen anytime in async-reset whereas effect of reset will happen only on the active edge of a clock in sync-reset.
- Async-reset is prone to glitches where as sync-resets are safer in this regard.
- Async-reset needs to meet only the minimum reset pulse width for FF where as sync-reset has to be long enough to be sampled on the next clock edge.

- Async-reset

input still needs the double FF synchronization to avoid race condition during reset de-assertion, while sync-reset do not need such additional circuitry.

What logic gets synthesized when integer is used instead of a reg variable as a storage element?

When

an integer is used as a storage element in a Verilog RTL code, it gets synthesized as a block of flip-flops similar to when a `reg` variable is used. Note that synthesis tool will optimize and remove unused flops.

```
module design (...);
  integer tmp;
  reg [7:0] data;

  always @ (posedge clk) begin
    // tmp [31:8] will be optimized because it is unused
    // since width of "data" is only 8b and so "tmp" does
    // not ever need anything more than 8b.
    tmp <= data;
  end
endmodule
```

However,

the encoding and implementation of the flip-flops may differ between integers and regs, and this may affect the timing, area, and power consumption of the design. `reg` the recommended storage element for most RTL designs, as they are optimized for sequential logic operations and have predictable timing, area, and power consumption characteristics.

Read more on [Verilog Data Types](#).

How do I choose between a case statement and a multi-way if-else statement?

A `case` statement should be chosen to implement a multiplexer, whereas a multi-way `if-else` statement should be chosen to implement a priority encoder.

Note that if the `default` clause is missing in a `case` statement or all the possible cases are not specified then a latch is inferred. Similarly, for an `if-else` construct, a missing final `else` clause will infer a latch.

Read more on [Verilog case](#) and [if-else-if](#) statements.

How do I avoid a priority encoder in an if-else tree?

The SystemVerilog keyword `unique` can be used which indicates that the order of decisions is not

important and it would synthesize into parallel logic or multiplexer.

```
unique if (in[0]) sel = 0;  
else if (in[1]) sel = 1;  
else sel = 2;
```

Read more on [SystemVerilog 'unique' and 'priority' case](#).

What are the differences between if-else and the ?: conditional operator?

- Conditional operator is typically used in continuous assignments while `if-else` is used within procedural blocks.
- A true and false expression is always required for the conditional operator whereas `else` part is optional in the `if` construct.
- Conditional operators cannot have block statements with `begin` and `end` whereas `if` conditions can.

- Deeply nested conditional expressions are harder to understand whereas `if` statements are cleaner in this regard.

Read more on [Verilog Conditional Statements](#).

What is the importance of a default clause in a case construct?

The `default`

clause is executed when none of the other cases are satisfied and if this is missing the logic will have to remember what the earlier value was and hence synthesis will infer a latch.

What is the difference between full_case and parallel_case synthesis directive?

`full_case` Indicates that the case statement has been fully defined and all unspecified case items can be optimized by the synthesis tool. `parallel_case` Indicates that all case items need to be evaluated in parallel and not infer any priority encoders. Avoids latch as all cases are defined. Results in multiplexer logic `default` clause can be avoided and still not infer a latch, although its not recommended to do so. Priority encoder is not synthesized as each path is unique.

Read more on [Verilog case statement](#).

What

is the difference in implementation with sequential and combinatorial processes, when the final else clause in a multiway if-else construct is missing?

In a sequential process, if the final else clause is missing, the synthesis tool will generate a synthesis warning or error since it indicates a missing case. This is because, in a sequential process, when the state machine is in a particular state, it

must always navigate to a subsequent state. An incomplete if-else statement violates this requirement and could lead to unpredictable behavior in sequential circuits. Therefore, it is essential to add a final else clause that catches all cases, explicitly updating the state or initiating a defined process before it returns to the beginning to avoid unexpected behavior.

In contrast, in a combinatorial process, the absence of a final else clause is not considered an error since the logic is always evaluated and computed afresh without relying on any past history or state. The expression is evaluated continuously and only depends on the values of the inputs to the expression. The synthesis tool will infer a latch or memory element to store the previous value, and it uses this to calculate the output at each step.

What

is the difference in using (== or !=) vs. (=== or !==) in decision making of a flow control construct in a synthesizable code?

Only

logical equality/inequality operators (== and !=) are synthesizable. Case equality (===) and inequality (!==) operators also compare X and Z values, whereas result is always false for logical comparison.

When

both operands have unknown or high-impedance state 'x' as one of the bits, the regular equality and inequality operators will return 0, indicating a false result. However, in case operators, this comparison will generate an 'x' or unassigned, indicating that the inputs are not predictable.

```
// If a or b has X/Z values (in addition to a!=b),  
// out will be driven a OR b  
if (a == b)  
    out = a & b;  
else  
    out = a | b;
```

```
// Both a and b match is done for 4-states  
// out = a AND b only when a/b are both 0, 1, X or Z  
if (a === b)  
    out = a & b;  
else  
    out = a | b;
```

- • What are the considerations to be taken choosing between flop-flops vs. latches in a design?
- • Which one is better, asynchronous or synchronous reset for the storage elements?
- • What logic gets synthesized when integer is used instead of a reg. variable as a storage element?
- • How do I choose between a case statement and a multi-way if-else statement?
- • How do I avoid a priority encoder in an if-else tree?
- • What are the differences between if-else and the ?: conditional operator?
- • What is the importance of a default clause in a case construct?
- • What is the difference between full_case and parallel_case synthesis directive?
- • What is the difference in implementation with sequential and combinatorial processes, when the final else clause in a multiway if-else construct is missing?
- • What is the difference in using (== or !=) vs. (=== or !==) in decision making of a flow control construct in a synthesizable code?

1. **Functional requirements:** The first consideration is the functional requirements of the system.

Different types of flip-flops and latches have their specific functions and capabilities, and it is necessary to choose the appropriate type based on the system's requirements.

2. **Timing requirements:**

Flip-flops have a fixed clock edge at which they latch the data, while latches hold the data as long as the enable signal is active. Latches facilitate time borrowing or cycle stealing, and helps increase pipeline depth with lesser area.

3. **Power consumption:** Flip-flops

tend to consume more power than latches, as they operate continuously

with a clock signal. If power consumption is a concern, latches may be preferred.

4. **Noise immunity:** Flip-flops are more immune to noise than latches since they latch their values at a specific clock edge. If the design has a lot of noise, flip-flops may be the better option.
5. **Area and cost:** Latches are typically smaller and less expensive than flip-flops. If the design needs to optimize for area or cost, latches may be preferred.

6. **Operating frequency:**

With time borrowing and cycle stealing, operating frequency is higher than the slowest logic path for latch. But in the case of FF, the slowest path pretty much decides the operating frequency.

- Reset signal is not part of the data path in async-reset whereas reset signal is part of the D input of the FF in sync-reset.
- Effect of reset can happen anytime in async-reset whereas effect of reset will happen only on the active edge of a clock in sync-reset.
- Async-reset is prone to glitches whereas sync-resets are safer in this regard.
- Async-reset needs to meet only the minimum reset pulse width for FF whereas sync-reset has to be long enough to be sampled on the next clock edge.
- Async-reset input still needs the double FF synchronization to avoid race condition during reset de-assertion, while sync-reset does not need such additional circuitry.
- Conditional operator is typically used in continuous assignments while `if-else` is used within procedural blocks.
- A true and false expression is always required for the conditional operator whereas `else` part is optional in the `if` construct.
- Conditional operators cannot have block statements with `begin` and `end` whereas `if` conditions can.
- Deeply nested conditional expressions are harder to understand whereas `if` statements are cleaner in this regard.

[Interview Questions](#) [Verilog Interview Set 1](#) [Verilog Interview Set 2](#)
[Verilog Interview Set 3](#) [Verilog Interview Set 4](#) [Verilog Interview Set 5](#)
[Verilog Interview Set 6](#) [Verilog Interview Set 7](#) [Verilog Interview Set 8](#)
[Verilog Interview Set 9](#) [Verilog Interview Set 10](#) [Verilog Interview Set 11](#)
[Verilog Interview Set 12](#) [Verilog Interview Set 13](#) [Verilog Interview Set 14](#)
[Verilog Interview Set 15](#) *Poll How would you rate the overall quality of this website ?* ★ ★★ ★★★★★ ★★★★★★ Related
[Topics](#) [Digital Fundamentals](#) [Verilog Tutorial](#) [Verification](#) [SystemVerilog Tutorial](#) [UVM Tutorial](#) [How can we improve ?](#)

▼ Verilog Interview Questions Set 12

- Explain the differences and advantages of casex and casez over the case statement?
- What are the differences between synchronous and asynchronous state machines?
- Illustrate the differences between Mealy and Moore state machines.
- Illustrate the differences between binary encoding and one-hot encoding mechanisms state machines.
- Explain a reversed case statement, and how it can be useful to infer a one-hot state machine?
- Illustrate how a multi-dimensional array is implemented.
- What are the considerations in instantiating technology specific memories?
- What are the factors that dictate the choice between synchronous and asynchronous memories?
- What are some reusable coding practices for RTL Design?
- What are snake paths, and why should they be avoided?

Explain the differences and advantages of casex and casez over the case statement?

casex has to be used when both X and Z needs to be treated as don't care for comparisons with the case item. **casez** on the other hand only treats Z as

don't care.

```
case (abc)
  3'b000 : out = a & b;    // same as 3'b000 and 3'b100
  3'b100 : out = a | b;    // same as 3'b100 and 3'b101
  default : out = ~(a & b); // for cases where bits in abc can be X or Z
endcase
```

Here are a couple of advantages:

- **Synthesis optimization:** `case` and `casez` can be optimized more effectively by synthesis tools than the `case` statement. This is because `case` and `casez` allow for more efficient encoding of the match conditions, reducing the number of gates required to implement the logic.
- **Code readability:** `case` and `casez` can make Verilog code more readable and concise. This is because they allow for more complex matching logic to be expressed in a single statement, rather than requiring multiple if-else statements.

What are the differences between synchronous and asynchronous state machines?

Synchronous

state machines have a clock input that triggers state transitions at specific times. When the clock signal rises, the state machine updates its output values and transitions to the next state based on its current state and input values.

Asynchronous state machines do not

require a clock signal as they are triggered by input signals that are not synchronized in time. Each input signal can trigger a state transition at any time, independent of any clock signal. Asynchronous state machines can be more complex to design and test due to the possibility of race conditions and glitches, but they can be more efficient and consume less power compared to synchronous state machines.

Illustrate the differences between Mealy and Moore state machines.

Mealy State Machine:

- Outputs are a function of both current state and inputs.
- Output may not be stable for one clock cycle as it is a function of input and current state.
- Output is prone to glitches.
- State transitions are based on both the current state and input signals.
- If inputs are not registered, combinational paths could potentially be larger than Moore machine, less operating frequency.

Moore State Machine:

- Output is based solely on the current state
- Output is stable for one clock cycle.
- Output is not prone to glitches.
- State transitions are based solely on the current state.
- Combinational paths are typically shorter with no involvement of inputs.

Illustrate the differences between binary encoding and one-hot encoding mechanisms state machines.

In *binary encoding*,

each state is represented by a unique binary code, where each bit of the code represents a possible state in the state machine. For example, if a state machine has four possible states, it can be represented using two bits (00, 01, 10, and 11).

- Binary encoding uses fewer flip-flops compared to one-hot encoding, which makes it more efficient in terms of hardware utilization.
- Timing is not as good as one-hot encoding due to more combinatorial paths
- Requires more effort for maintenance and debug

- Usually preferred in ASICs unless output path timing is critical

In *one-hot encoding*,

each state is represented by a unique bit, with only one bit set to 1 and all other bits set to 0. For example, in a state machine with four possible states, each state is represented by a unique combination of four bits, with only one bit set to 1.

- The state transitions are clearly defined, which makes it easier to detect errors and glitches in the state machine
- One-hot encoding has a faster response time compared to binary encoding since only one bit changes state at a time and there's only clk→q delay
- It is easier to design and debug state machines using one-hot encoding
- Useful in register rich applications like FPGAs

Explain a reversed case statement, and how it can be useful to infer a one-hot state machine?

A `case` expression can be a constant used to match against a variable also.

```
// 4-bit variable to store 4 states in one-hot encoding
reg [3:0] cur_state, next_state;

case (1'b1)
  cur_state[0] : // Assign next state
  cur_state[1] : // Assign next state
  cur_state[2] : // Assign next state
  cur_state[3] : // Assign next state
endcase
```

Illustrate how a multi-dimensional array is implemented.

Synchronous

static memories that can be used like a register file can be defined as a multi-dimensional array which then gets synthesized into FFs. For the

example below, each row of memory requires 8 FF, and with a depth of 16, the total FFs required will be 128.

```
parameter DEPTH = 16;
parameter WIDTH = 8;

reg [WIDTH-1:0] mem [DEPTH-1:0];

always @ (posedge clk) begin
    if (wr)
        mem[addr] <= i_data;
    else
        o_data <= mem[addr];
end
```

Using

a hardmacro of memory from a semiconductor vendor with better area and power would be better than synthesizing it using discrete logic.

What are the considerations in instantiating technology specific memories?

When

instantiating technology-specific memories, there are several key considerations to keep in mind to ensure optimal performance and efficiency:

- **Area:** A high density memory would be required to reduce area footprint of the design on the die, typically used by chips with large memory blocks.
- **Frequency:** Special high-speed memory cells that operate at high frequencies may be required if speed is the main concern, but it may have larger area.
- **Power requirements:** The power consumption of the memory should be considered, especially in low-power applications. Memories such as DRAM and SRAM typically require low power, while NAND flash memory requires higher power.

- **Memory type:** Depending on the application, the appropriate memory technology should be selected, such as SRAM, DRAM, or flash memory, each with their own advantages and disadvantages.
- **Memory access speed:** The memory access speed should be chosen based on the performance requirements of the system. DDR memory provides high-speed access, whereas NOR flash memory provides fast read access.
- **Pinout and package options:** The pinout and package options should be selected based on the available space and required signal requirements of the system.
- **Single vs Multi Port:** Depends on overall design architecture required to support a targeted performance of the system. Multiple ports allow write and read transactions to happen concurrently thereby boosting its performance.
- **Voltage requirements:** The voltage requirements of the memory should also be taken into account to ensure compatibility with the power supply and other components in the system.
- **Cost:** The cost of the memory should be evaluated against the benefits provided to ensure that it is a cost-effective solution.

What are the factors that dictate the choice between synchronous and asynchronous memories?

- **Performance:** Synchronous memories are generally faster than asynchronous memories and synchronize data transfer with a clock signal. For applications that require high-speed data transfer, synchronous memory is the better choice.
- **Latency:** Asynchronous memories typically have higher read and write latencies whereas synchronous memories have predictable latencies and faster cycle times. Asynchronous memories

may

be acceptable for applications that can tolerate higher latencies.

- **Power Consumption:** Asynchronous memories typically consume less power compared to synchronous memories since there's no internal clock signal making them desirable for power-sensitive applications.
- **Timing:** Synchronous memories have better static timing because data output is registered with a FF, whereas asynchronous memories have combinatorial paths that may become critical element in the timing path.
- **Area:** Synchronous memories require more area compared to asynchronous memories.
- **Noise immunity:** Synchronous memory has better noise immunity compared to asynchronous memory. The use of the clock signal makes it less vulnerable to electrical noise and other forms of interference.
- **Cost:** Asynchronous memories are less complex and easier to manufacture, making them more cost-effective compared to synchronous memories.

What are some reusable coding practices for RTL Design?

- Better timing closure can be achieved if all outputs of critical design blocks are registered.
- Avoid snake paths which make debugging tedious and synthesis difficult.
- Avoid instantiation of technology specific gates.
- Use parameters and macros instead of hard-coded values in design RTL.
- Breaking down functionality into smaller submodules can help to minimize the complexity of your code, which makes it more reusable.

- Use generic RTL code is general-purpose code that can be reused in many different applications.
- Develop your coding style to facilitate readability and maintenance. Consistent coding practices, such as standard naming conventions and formatting, can make your code more understandable and reusable.
- Intellectual property (IP) modules are proven, tested designs that have been created for reuse. Reusing IP modules can minimize the development time, cost, and risk associated with designing from scratch.
- Utilizing a standardized library with commonly used functions makes it easier to design and maintain RTL code across multiple projects.

What are snake paths, and why should they be avoided?

It

is a path that goes through different levels of hierarchies and return to the same level where it started from. These should be avoided because it will have a long timing path and may turn out to be a critical path in chip top, and synthesis tools will have harder time to constrain paths leading to larger runtimes.

To avoid this:

- Register the outputs of modules with different functional objectives
- Partition the design functionality, and aim for short and direct routes between components.

▼ Verilog Interview Questions Set 13

- What are a few considerations while partitioning large designs?
- How can I reliably convey control information across clock domains?
- What is a safe strategy to transfer data of different buswidths and across different clock domains?

- What are a few considerations while using FIFOs for posted writes or prefetched reads that influence the speed of the design?
- What will be synthesized of a module with only inputs and no outputs?
- Why do I see latches in my synthesized logic?
- What are combinatorial timing loops? Why should they be avoided?
- How does the sensitivity list of a combinatorial always block affect pre- and post- synthesis simulation?
- How do the `ifdef, `ifndef, `elsif, `endif constructs aid in minimizing area?
- What is constant propagation? How can I use constant propagation to minimize area?

What are a few considerations while partitioning large designs?

- **Size and complexity of the design:** A large design will need to be partitioned into a number of smaller designs. This can affect how the design is divided into different sections and the size of each partition.
- **Clock Domains:**
It is recommended to group logic belonging to same clock domain in a single block, and clock domain crossings done thorough a synchronizer.
- **Specific design requirements:** Specific design requirements, such as timing or power constraints, will affect how the design is partitioned.
- **Vendor's requirements:** The vendor's requirements must also be considered as the partitioning of designs will be determined largely by their manufacturing capabilities.

How can I reliably convey control information across clock domains?

- **Use a two-flop synchronizer:** A two-flop synchronizer is a common technique used to safely transfer data between clock domains. It consists of two registers placed in

series, one in each clock domain, to ensure reliable transfer of data across domains.

- **Be aware of clock skew:** Clock skew can occur between different clock domains and can adversely affect the timing of the control signal. To mitigate this, compensate for the clock skew by adding a delay buffer.
- **Consider using asynchronous FIFOs:** Asynchronous FIFOs are used to transfer data between clock domains that have different clock frequencies. By implementing flow control and arbitration logic, asynchronous FIFOs can help avoid data loss, blocking, or lock-up.
- **Simulate and verify:** Always simulate and verify the design with all possible corner-case scenarios to ensure that control information is reliably transferred across different clock domains.

What is a safe strategy to transfer data of different buswidths and across different clock domains?

An

asymmetrical FIFO is an ideal component for this purpose where the bus width between the write side and the read sides are different.

Read more on [Synchronous FIFO](#).

What are a few considerations while using FIFOs for posted writes or prefetched reads that influence the speed of the design?

When

the write and read sides are running on different clock frequencies, a FIFO acts as a buffer thereby allowing the write side to simply push the transfer into the buffer instead of waiting for the slave to respond and the transfer to complete. This allows better utilization of the bus.

However,

assume that the master posted data into the FIFO and assumed it to have completed. If the slave now issues an error, this condition will have

to be routed back to the master somehow like an interrupt so it can resend the same transfer.

If the master aborts a read transaction late in the cycle when read prefetch has already taken place, then it is possible to get stale data from the FIFO, and contents may need to be flushed.

What will be synthesized of a module with only inputs and no outputs?

If a module has only inputs and no outputs, then the synthesis tool may throw empty modules with no logic or little logic.

Why do I see latches in my synthesized logic?

Latches

usually result when the tool cannot determine a unique value for a signal during synthesis. If the sequential logic in your RTL code is incomplete, contains uninitialized values, or does not have a defined output value for some input combination, the synthesis tool may generate latches to store the input signals' present values. If there are incomplete `case` or `if else` statements in the RTL code, the synthesis tool may generate latches to hold intermediate states until a valid condition is met.

Read more on [Latch](#).

What are combinatorial timing loops? Why should they be avoided?

Combinatorial

timing loops, also known as combinational feedback loops, are a type of timing issue that can occur in digital circuits. They happen when a logic path's output is fed back to its input without passing through any registers or flip-flops, creating an infinite loop that prevents the circuit from settling to a stable state. This means that the output of the circuit is undefined.

```
// A simple example shown below has output dependent on itself
// However, it can be complex with more logic in between but ultimately
```

```
// feeding back to the same signal without any flops in between
assign out = out & in;
```

Combinatorial timing loops should be avoided in digital circuits for a few reasons:

- **They can cause glitches:** When a combinational loop exists, it creates a race condition where the output of the loop is dependent on the delays of the logic gates. Any small variation in these delays may cause the output of the circuit to glitch, leading to unpredictable behavior.
- **They increase power consumption:** When a combinational loop exists, the logic gates are continuously toggling outputs without settling to a stable state. This results in a high switching activity, which can increase the power consumption of the circuit.
- **They increase the propagation delay:** Combinatorial loops can lead to an increase in the circuit's propagation delay since the output of the loop depends on the previous output.

How does the sensitivity list of a combinatorial always block affect pre- and post- synthesis simulation?

The sensitivity list of a combinatorial always block specifies the input signals to the block that trigger its execution. The sensitivity list informs the simulation tool which input signals to monitor for changes that would require an update in the output.

Verilog and SystemVerilog languages have evolved to allow designers represent their intentions more easily with new constructs.

```
// A normal sensitivity list should include all signals that affect output
always @ (a, b, c, d) begin
    out = (a & b) ^ (c | d);
end

// Use * to let tool automatically add such signals into the sensitivity list
```

```

always @ (*) begin
    out = (a & b) ^ (c | d);
end

// SystemVerilog has another keyword
always_comb begin
    out = (a & b) ^ (c | d);
end

```

How do the `ifdef, `ifndef, `elsif, `endif constructs aid in minimizing area?

The conditional compilation constructs, such as `ifdef, `ifndef, `elsif, and `endif, allow the designer to choose which part of the design is included in the synthesis process. This is useful when certain parts of the design are not necessary for the final implementation, and including them can result in an unnecessarily large area. By selectively including parts of the design, the synthesis tool can optimize the output circuit for a smaller size.

Read more on [Verilog `ifdef Conditional Compilation](#).

What is constant propagation? How can I use constant propagation to minimize area?

Constant propagation is a compiler optimization technique that replaces the usage of variables with their constant values which helps reduce area and improve the performance of the synthesized circuit.

```

parameter zero = 0;

assign out = (zero == 1) ? in1 : in2;

```

In Verilog, constant propagation optimizes the circuit by replacing the values of variables that remain constant throughout the simulation with their final values. If a variable has a constant value, the synthesis

tool optimizes the logic by replacing the variable with a wire that has the same constant value. This can help reduce the size of the synthesized circuit since fewer resources are required to implement the design.

▼ Verilog Interview Questions Set 14

- What happens to the bits of a reg which are declared, but not assigned or used?
- How does the generate construct help in optimal area ?
- How can you ensure the synthesizable Verilog will build a sequential logic ?
- Can the generate construct be nested?
- What is a critical path in a design? What is the importance of understanding the critical path?
- How does proper partitioning of design help in achieving static timing?
- What does it mean to "retime" logic between registers? How does it effect functionality?
- Why is one-hot encoding preferred for FSMs designed for high-speed designs?
- What are the main factors that affect testability of a design?
- My chip has on-chip tri-state buses. What are the testability implications, and how do I take care of it?

What happens to the bits of a reg which are declared, but not assigned or used?

Bits that are unused are optimized away during synthesis.

```
module design (...);  
  
reg [3:0] tmp;  
  
always @ (posedge clk) begin
```

```

if (! resetn) begin
    tmp <= 0;

    // Since tmp[1] and tmp[2] are unused,
    // synthesized logic will contain only 2 flops
end else begin
    tmp[0] <= in;
    tmp[3] <= ~in;
end
end

endmodule

```

How does the generate construct help in optimal area ?

The `generate` construct is a feature that allows designers to create multiple instances of a module or block of code.

It allows the use of a variable using `genvar` keyword to control the generated logic. It can have `for` loop, `if else` or `case` statements inside it for conditional generation of replicated hardware.

Read more on [Verilog generate block](#).

How can you ensure the synthesizable Verilog will build a sequential logic ?

- Use non-blocking assignments in `always` block triggered @ `posedge clk`
- Use positive edge triggered FF for state
- Do not assign the same variable in multiple always blocks
- Keep sequential and combinational blocks separate
- Do not mix blocking and non-blocking assignments in the same procedural block

Can the generate construct be nested?

No, the `generate` construct cannot be nested and will result in a syntax error if it is done so. But, `if else`, `case` conditional statements and `for` loops can be nested.

```
module design (...);

generate
    ...
    // Not allowed, results in error
    generate
        ...
    endgenerate
endgenerate

// Same module can have multiple independent
// generate blocks
generate
    ...
endgenerate

generate
    ...
endgenerate

endmodule
```

Read more on [Verilog generate block](#).

What is a critical path in a design? What is the importance of understanding the critical path?

In

a digital design, the critical path is the path of logic gates and interconnects that has the longest propagation delay, limiting the overall maximum operating frequency of the circuit. It is the slowest path of the combinational logic circuits that has the most significant impact on the performance of the design.

Accurately identifying

the critical path is essential for timing analysis and avoiding timing violations, which can cause synchronization issues, glitches, or failures. Once the critical path is known, the designer can optimize it through techniques like pipelining, parallelism, or parallel paths to improve performance without affecting the design's functionality.

How does proper partitioning of design help in achieving static timing?

Partitioning

the design refers to dividing a complex design into smaller sub-blocks that can be designed and analyzed separately to simplify the overall design.

Here are a few ways proper partitioning of a design can help:

- **Reduces the complexity of timing analysis:** Breaking down the design into smaller sub-blocks reduces the complexity required for the Static Timing Analysis (STA) and enables timing closure, making it easier to identify and fix the bugs.
- **Enables parallel processing:** Partitioned designs can be processed in parallel, thereby reducing the overall computation time. The designer can then optimize each sub-block independently for maximum utilization of computational resources.
- **Enables design reuse:** Partitioning the design allows for the reuse of the smaller sub-blocks in different designs, leading to reduced development time and cost.
- **Reduces the critical path:** Partitioning the design and optimizing each sub-block reduces their critical paths, ensuring multiple non-critical paths, which lowers the system's worst-case path delay and helps meet timing constraints.

What does it mean to "retime" logic between registers? How does it effect functionality?

Retiming

is a technique used in digital circuit design to balance the path delay

between sequential elements (registers) and combinational logic circuits. Retiming involves moving flip-flops so that the critical path is shifted to reduce its delay.

When correctly applied, retiming does not affect the functionality of the circuit. The circuit maintains its intended behavior, but the delay between the registers is redistributed, allowing for improvements in overall performance.

Why is one-hot encoding preferred for FSMs designed for high-speed designs?

One-hot

encoding is a technique used to encode state bits of a Finite State Machine (FSM) such that only one state bit is high (1) for each state. This encoding method is particularly useful for high-speed FSM designs due to the following reasons:

- **Reduced delay and fast transition:** In a one-hot encoded FSM, each state can be represented by a single flip-flop, and a change of state requires changing the output of only one flip-flop which reduces propagation delay and ensures fast state transition times.
- **Concurrent output generation:** Each state corresponds to a unique output bit, so concurrent output generation is possible. This is because the current state can be determined without the propagation delay of a decoder, which enables faster clock speeds, improved throughput, and smaller critical paths.
- **Easier timing closure:** It makes the design simpler and less prone to timing violations which results in an easier timing closure .
- **Reduced area and power consumption:** Using a one-hot encoded FSM allows for easier implementation and requires fewer gates in the design, resulting in reduced area and power consumption.

What are the main factors that affect testability of a design?

Some factors include the presence of derived and gated clocks, latches and tri-state buses in the design.

My chip has on-chip tri-state buses. What are the testability implications, and how do I take care of it?

Tri-state

buses allow multiple devices to share data lines while minimizing the capacitance and current requirements for each device. The key issue with testing tri-state buses is to ensure that any open or stuck gates associated with the tri-state buses can be identified and tested effectively. These faults can cause contention or isolation errors, leading to intermittent or erratic behavior of the chip.

Some ways to improve testability are:

- **Controlling the tri-state gates:** To test the tri-state buses, it is essential to ensure that one device at a time is driving the bus. This can be achieved by controlling the tri-state gates and ensuring that all but one device is in the high-impedance state during the test.
- **Using built-in self-test (BIST) circuitry:** BIST circuitry can be added to the design to control and isolate the tri-state buses for testing. BIST circuitry can be used to scan in test patterns that will isolate and exercise each tri-state gate within the design.

▼ Verilog Interview Questions Set 15

- Async resets of some FFs are driven by other FFs, how does this affect testability ?
- What are the testability implications for derived clocks ?
- What are the testability challenges of gated clocks ?
- What is the implication of a combinatorial feedback loops in design testability?
- How does the presence of latches affect the testability ?

- What value is sampled by the logic from an input port that is left open (that is, no-connect) during its module instantiation?
- How is the connectivity established in Verilog when connecting wires of different widths?
- Can I use a Verilog function to define the width of a multi-bit port, wire, or reg type?
- Summarize the main differences between \$strobe and \$monitor
- How can I selectively enable or disable monitoring?

Async resets of some FFs are driven by other FFs, how does this affect testability ?

During scan testing, if the driving FF gets a pattern such that it resets the driven FF, it will lose its data. Hence the *reset* output from the first FF should be OR'd with a test mode enable signal before assigning to the second FF.

What are the testability implications for derived clocks ?

Derived

clocks are created within a chip using clock dividers through Flip-Flops or PLLs. Because these clocks are generated internally, a control signal from the primary pins is necessary to prevent Flip-Flops from capturing data at unintended times.

A multiplexor can be

added in the clock path with the select line controlled by a test mode enable signal. The inputs to the mux should be regular clock and derived clock.

What are the testability challenges of gated clocks ?

Gated

clocks are typically used to reduce power consumption by disabling the clock signal to specific parts of the chip when they are not in use.

However, during testing, it is essential to be able to access and

control these signals in order to ensure correct operation of the chip and detect any faults.

The workaround is to logically OR a test mode enable signal to the enabling pin of the AND gate that gates the clock.

What is the implication of a combinatorial feedback loops in design testability?

Combinatorial

feedback loops occur when the output of a combinational logic gate is fed back into one of its inputs, potentially creating a feedback loop that can lead to unexpected behavior or cause the design to become stuck in an unstable state. Since the loops are delay-dependent, they cannot be tested with any ATPG algorithm and must be avoided in the logic.

How does the presence of latches affect the testability ?

One

of the main issues with latches is that they can cause the propagation of glitches, which are short-lived pulses that can be difficult to observe during testing. These glitches can be especially problematic if they propagate to other parts of the design, potentially causing unintended behavior or creating false positives during testing.

Output

of a latch is not controllable directly from a primary input since, the enable to a latch is not the regular clock going to rest of the flops in the design. The enable pin to the latch needs to be OR'd with a test mode enable signal.

What value is sampled by the logic from an input port that is left open (that is, no-connect) during its module instantiation?

An

unconnected input port is a floating port, and it may float to an intermediate voltage level due to parasitic capacitance and inductance in the circuit. In simulations, this is indicated by the value 'Z' or

high impedance and the logic following it will also propagate the 'Z' until gated off by an AND gate.

To avoid these types of issues, it is generally recommended to tie all unused input ports to a known state, such as ground or VDD, through the use of pull-up or pull-down resistors. This can help to prevent the input signal from floating and ensure that the logic gates are working as intended.

The default value of Z can be changed using compiler directives.

```
// Causes all unconnected input ports following this to be pulled down to logic 0
`unconnected_drive pull0

module mod_2341( ... );
    ...
endmodule

// Do not apply for rest of the code
`nounconnected_drive
```

How is the connectivity established in Verilog when connecting wires of different widths?

Connections are right-justified, which means that the LSB of the RHS is connected to the LSB of the LHS.

```
wire [7:0] net1;
wire [3:0] net2;

// net2 has smaller width than net1, and net1[7:4] is left floating
assign net1 = net2;

// net2 gets the truncated value of net1[3:0]
assign net2 = net1;
```

Can I use a Verilog function to define the width of a multi-bit port, wire, or reg type?

No,

you cannot use a Verilog function to define the width of a multi-bit port, wire, or reg type. The width of a Verilog port, wire, or reg declaration must be a constant or a parameter value, both of which are determined at compile-time.

Functions, on the other hand, are executed at run-time and their return values cannot be used to dynamically set the widths of Verilog objects. Attempting to use a function to define the width of a port, wire, or reg will result in a compile-time error. However, you can use function parameters to define the width of Verilog objects. For example, you can declare a parameter and use it to specify the width of a port or signal as follows:

```
parameter WIDTH = 8;

input [WIDTH-1:0]    input_signal;    // Okay
input [find_width():0] input_signal;    // Error
```

Summarize the main differences between \$strobe and \$monitor

\$strobe

displays values of selected signals at the end of the current simulation time when all simulation events have occurred and just before time advances whereas **\$monitor** displays the value of selected signals whenever its value changes.

Read more on [Verilog Display Tasks](#).

How can I selectively enable or disable monitoring?

This can be done with **\$monitoron** and **\$monitoroff** tasks.